
Full-Waveform Inversion

Teoria e prática com PyFWI

Um curso completo, do problema direto ao problema inverso
com código executável e verificação numérica

“A FWI não tem válvula de escape: o que ela não consegue explicar pela física, ela explica mexendo no modelo.”

15 aulas executáveis · propagador próprio verificado · PyFWI em GPU

12 de setembro de 2026

Sumário

Como usar este material	4
Parte I — Física e numérica da propagação	7
1 O que é FWI, e por que é difícil	7
1.1 A formulação em uma linha	7
1.2 As três dificuldades que definem o campo	7
1.3 O que a FWI recupera bem, e o que não recupera	8
2 O regime acústico	9
2.1 Ponto de partida: elastodinâmica	9
2.2 A aproximação acústica: uma única hipótese	9
2.3 O que se preserva e o que se perde	11
2.4 A fonte: wavelet de Ricker	11
3 Diferenças finitas: estabilidade e dispersão	13
3.1 O operador espacial	13
3.2 Estabilidade: a condição CFL	13
3.3 Dispersão numérica: o erro que parece geologia	14
3.4 Receita de dimensionamento	14
4 Bordas, superfície livre e aquisição	16
4.1 Por que as bordas não são detalhe	16
4.2 Superfície livre: uma escolha física	17
4.3 Aquisição	17
Parte II — PyFWI	20
5 PyFWI na prática	20
5.1 A anatomia de um script	20

5.2	O parâmetro <code>components</code>	21
5.3	A prova numérica do regime acústico	21
5.4	Por que as formas diferem (e não é erro de nenhum dos dois)	21
5.5	Validação cruzada: como esta comparação achou um bug	23
6	PyFWI por dentro: parâmetros e custo	24
6.1	Referência do dicionário <code>inpa</code>	24
6.2	O reescalonamento automático de <code>dt</code>	25
6.3	Checkpointing: o compromisso central	25
6.4	A conta que evita surpresas	25
	Parte III — O problema inverso	28
7	Função objetivo e cycle skipping	28
7.1	Formulação	28
7.2	Vendo a não convexidade	28
7.3	Cycle skipping: a explicação física	29
7.4	A estratégia multiescala	30
7.5	O modelo inicial	30
7.6	Além do L2	30
8	O método do estado adjunto	32
8.1	Por que não diferenças finitas	32
8.2	A derivação	32
8.3	Da fórmula ao código	34
8.4	Como ler um gradiente	34
8.5	Pré-condicionamento	35
9	Verificação do gradiente	37
9.1	Por que este é o teste mais importante do seu código	37
9.2	Os três testes	37
9.2.1	Diferenças finitas pontuais	37
9.2.2	Derivada direcional	38
9.2.3	Teste de Taylor	38
9.3	Estudo de caso: o bug que estes testes encontraram	39

Parte IV — Otimização, FWI completa e regularização	42
10 Otimização	42
10.1 O panorama	42
10.2 A busca linear, onde o tempo é gasto	42
10.3 Comparação medida	43
11 A FWI completa: multiescala	45
11.1 A estratégia	45
11.2 Um experimento controlado	45
11.3 Estudo de robustez: e quando o inicial é ruim?	46
11.4 Como reportar uma FWI	47
12 Regularização	48
12.1 Por que regularizar	48
12.2 Tikhonov <i>versus</i> Variação Total	48
12.3 Um resultado que contraria a expectativa	49
12.4 Escolhendo λ	50
12.5 Modelo a priori e vínculos	50
Parte V — Os limites	52
13 Os limites da aproximação acústica	52
13.1 O inventário de hipóteses	52
13.2 Limite 1: densidade e a ambiguidade da impedância	52
13.3 Limite 2: elasticidade	53
13.4 Limite 3: atenuação e o fator Q	53
13.5 Como decidir qual física você precisa	55
A Checklist de diagnóstico	56
B Notação	58
C Referências	59

Como usar este material

Este documento é a parte teórica de um curso que tem duas metades inseparáveis. A outra metade são 15 programas em Python, um por aula, que executam, medem e desenham tudo o que aqui é derivado. Ler sem rodar deixa o material pela metade; rodar sem ler também.

A estrutura

Parte	Capítulos	Assunto
I	1–4	Física e numérica da propagação: de onde vem a equação, como discretizá-la sem estragar a física
II	5–6	PyFWI como ferramenta de produção
III	7–9	O problema inverso: função objetivo, estado adjunto, verificação
IV	10–12	Otimização, FWI completa, regularização
V	13	Onde a física adotada deixa de valer

Cada capítulo indica a aula correspondente com a marca [\[aula 07\]](#). O código está em `aulas/`, e a caixa de ferramentas compartilhada em `fwikit/`.

As duas ferramentas, e por que são duas

`fwikit.acustico` é um propagador acústico 2D escrito do zero em NumPy, com 486 linhas — metade delas comentário. Não usa GPU, não é rápido, e é inteiramente transparente: cada termo está visível. É com ele que derivamos e *verificamos* o gradiente adjunto.

PyFWI é uma implementação de produção em OpenCL: elástica, com CPML e checkpointing. É rápida e é uma caixa-preta. Serve para rodar inversões de verdade e como referência independente.

Aprender só o primeiro deixa você lento. Aprender só o segundo deixa você sem saber o que fazer quando o resultado sai errado — e ele vai sair.

Sobre os números deste documento

Todos os valores numéricos, tabelas e figuras deste PDF foram *gerados pelo código do curso*, executado no ambiente descrito no `README.md`. Nenhum número foi copiado de livro. Onde um resultado contraria a expectativa usual — e há pelo menos um caso assim, no Capítulo 12 — o texto discute o resultado obtido, não o esperado.

As fórmulas são verificadas, não apenas citadas

O repositório traz `testes/verificar_fisica.py`, com 29 verificações que confrontam cada fórmula usada aqui com sua derivação analítica ou com uma referência independente: os coeficientes de diferenças finitas contra os momentos de Taylor, o limite CFL contra a análise de von Neumann, a Ricker contra seu espectro analítico, as reduções elásticas, o gradiente adjunto contra diferenças finitas (teste de Taylor), os gradientes dos termos de regularização, e o modelo constant- Q .

Elas rodam em poucos minutos:

```
1 python testes/verificar_fisica.py
```

Parte I — Física e numérica da propagação

Capítulo 1

O que é FWI, e por que é difícil

1.1 A formulação em uma linha

Full-Waveform Inversion é um problema de otimização. Existe um dado sísmico observado $\mathbf{d}_{obs}$. Existe um operador de modelagem F que, dado um modelo de subsuperfície $\mathbf{m}$, produz um sismograma sintético $F(\mathbf{m})$. A FWI procura o modelo que torna os dois mais parecidos possível:

$$\mathbf{m}^* = \arg \min_{\mathbf{m}} J(\mathbf{m}), \quad J(\mathbf{m}) = \frac{1}{2} \|F(\mathbf{m}) - \mathbf{d}_{obs}\|^2. \quad (1.1)$$

O adjetivo *full-waveform* distingue o método da tomografia clássica: usa-se a forma de onda inteira — amplitude, fase, todos os eventos — e não apenas o tempo de chegada de algumas fases escolhidas. É daí que vem a resolução superior, e é daí que vêm todas as dificuldades.

1.2 As três dificuldades que definem o campo

A equação (1.1) é simples de escrever e brutal de resolver, por três razões independentes. O curso inteiro está organizado em torno delas.

1. **F é caro.** Cada avaliação é uma simulação completa da equação da onda, para cada tiro. Daí a Parte I: simular bem, e simular barato.
2. **J não é convexa.** Tem mínimos locais por toda parte. O fenômeno tem nome próprio — *cycle skipping* — e causa próxima: diferença de fase maior que meio período. É o Capítulo 7.
3. **$\mathbf{m}$ é gigante.** De 10^4 a 10^8 incógnitas. Isso exclui qualquer método que precise montar a Hessiana, e exclui o cálculo do gradiente por força bruta. É a Parte III.

A ideia que torna a FWI possível

Calcular $\partial J / \partial \mathbf{m}$ por diferenças finitas exigiria uma simulação *por parâmetro*. Com 10^6 parâmetros, seriam 10^6 simulações por iteração.

O **método do estado adjunto** obtém o gradiente inteiro com *duas* simulações por tiro — uma direta e uma adjunta — e o custo *não depende do número de parâmetros*. Sem essa ideia não existiria FWI. É o Capítulo 8.

1.3 O que a FWI recupera bem, e o que não recupera

Uma expectativa calibrada evita meses perdidos. A FWI reconstrói bem os comprimentos de onda *intermediários* do modelo. Ela não reconstrói:

- os **comprimentos de onda muito longos** (a tendência regional de velocidade) — isso tem de vir do modelo inicial, porque o dado sísmico não tem frequência baixa suficiente para restringi-los;
- o que está **abaixo da profundidade de penetração**, controlada pela abertura da aquisição (regra prática: metade a um terço do maior *offset*);
- estruturas menores que $\sim \lambda_{\min}/2$;
- qualquer coisa em **zona de sombra**, onde nenhuma onda passou.

Capítulo 2

O regime acústico

[aula 01]

Este capítulo responde à pergunta que define todo o resto: *qual equação estamos resolvendo, e o que foi jogado fora para chegar nela?*

2.1 Ponto de partida: elastodinâmica

Toda a sismologia de exploração começa com dois ingredientes: a segunda lei de Newton para um meio contínuo, e uma lei constitutiva.

$$\rho \frac{\partial v_i}{\partial t} = \frac{\partial \sigma_{ij}}{\partial x_j} + f_i, \quad (2.1)$$

$$\frac{\partial \sigma_{ij}}{\partial t} = \lambda \delta_{ij} \frac{\partial v_k}{\partial x_k} + \mu \left(\frac{\partial v_i}{\partial x_j} + \frac{\partial v_j}{\partial x_i} \right). \quad (2.2)$$

Aqui v_i é a velocidade de *partícula* (não a velocidade de onda), σ_{ij} o tensor de tensões, ρ a densidade, e λ, μ os parâmetros de Lamé. O módulo de cisalhamento μ é o que resiste a deformação sem mudança de volume.

Em 2D, (2.1)–(2.2) formam um sistema de cinco equações acopladas nas incógnitas $(v_x, v_z, \sigma_{xx}, \sigma_{zz}, \sigma_{xz})$. **É exatamente esse sistema que o *kernel* OpenCL do PyFWI resolve** — volte a este ponto no Capítulo 5.

Do sistema elástico saem duas velocidades de onda:

$$v_p = \sqrt{\frac{\lambda + 2\mu}{\rho}}, \quad v_s = \sqrt{\frac{\mu}{\rho}}. \quad (2.3)$$

2.2 A aproximação acústica: uma única hipótese

A aproximação acústica consiste em **uma** hipótese física:

$$\boxed{\mu = 0 \iff v_s = 0} \quad (2.4)$$

isto é, o meio não resiste ao cisalhamento. Literalmente o comportamento de um fluido. As consequências são imediatas:

1. $\sigma_{xz} = 0$: não existe tensão cisalhante;
2. $\partial_t \sigma_{xx} = \partial_t \sigma_{zz} = \lambda (\partial_x v_x + \partial_z v_z)$: as duas tensões normais passam a obedecer à *mesma* equação. Como partem de zero, permanecem iguais em todo instante — $\sigma_{xx} = \sigma_{zz}$ — e o tensor vira isotrópico;
3. um tensor isotrópico é descrito por um único escalar. Definimos a **pressão** como esse escalar, com sinal trocado:

$$p = -\frac{\sigma_{xx} + \sigma_{zz}}{2} = -\sigma_{xx} = -\sigma_{zz}. \quad (2.5)$$

Guarde a equação (2.5)

É exatamente o que o PyFWI devolve quando você pede `components=0`: ele retorna `(taux + tauz)/2`, ou seja $-p$. O PyFWI resolve o sistema *elástico* e obtém o caso acústico como o limite $v_s \rightarrow 0$. O Capítulo 5 comprova isso numericamente.

Com $\mu = 0$, λ passa a ser o módulo de compressibilidade K , e $v_p = \sqrt{K/\rho}$. O sistema de cinco equações colapsa em três:

$$\rho \frac{\partial \mathbf{v}}{\partial t} = -\nabla p, \quad \frac{\partial p}{\partial t} = -K \nabla \cdot \mathbf{v} + s(t) \delta(\mathbf{x} - \mathbf{x}_s). \quad (2.6)$$

Eliminando $\mathbf{v}$, chega-se à forma de segunda ordem:

$$\frac{1}{K} \frac{\partial^2 p}{\partial t^2} = \nabla \cdot \left(\frac{1}{\rho} \nabla p \right) + s, \quad (2.7)$$

e, supondo densidade constante,

$$\boxed{\frac{1}{c(\mathbf{x})^2} \frac{\partial^2 p}{\partial t^2} - \nabla^2 p = s(t) \delta(\mathbf{x} - \mathbf{x}_s)} \quad (2.8)$$

A equação (2.8) é a que o `fwikit.acustico` resolve, e a mesma da formulação clássica de FWI acústica (Tarantola, 1984).

O s das duas equações não é o mesmo

Quase todo texto omite isto, e custa horas de depuração. Ao derivar $\partial_t p$ no tempo para eliminar $\mathbf{v}$, o termo-fonte de (2.6) vira $(1/K) \partial_t s$. Escrever “+ s ” em (2.7) e (2.8) é *redefinir* a fonte — prática padrão, mas com consequência concreta:

a wavelet efetiva das duas formulações difere por uma derivada temporal.

É por isso que dois códigos **corretos** — um em velocidade–tensão (1ª ordem, como o PyFWI) e outro em pressão (2ª ordem, como o `fwikit`) — produzem traços de forma diferente alimentados pela mesma Ricker. O Capítulo 5 mede isso: a correlação bruta entre os dois é de +0,12, e sobe para +0,85 depois de derivar um deles no tempo.

Por que invertemos em $m = 1/c^2$, e não em c

A equação (2.8) é *linear* no parâmetro $m(\mathbf{x}) = 1/c(\mathbf{x})^2$ — a vagarosidade ao quadrado — e não é linear em c .

Isso não é preferência estética: torna a derivada em relação ao parâmetro trivial, e o gradiente adjunto sai limpo (Capítulo 8). A conversão para velocidade é a regra da cadeia:

$$\frac{\partial J}{\partial c} = \frac{\partial J}{\partial m} \cdot \frac{dm}{dc} = \frac{\partial J}{\partial m} \cdot \left(-\frac{2}{c^3}\right).$$

2.3 O que se preserva e o que se perde

Fenômeno	Acústico	Comentário
Tempo de trânsito P	preservado	é o que a FWI mais usa
Difração / <i>multipath</i>	preservado	vantagem sobre tomografia de raio
Reflexão (baixo ângulo)	aproximado	coeficiente razoável
Reflexão (alto ângulo)	errado	AVO exige v_s
Onda S	ausente	não existe no modelo
Conversão P–S	ausente	contaminante em dado terrestre
Onda de superfície	ausente	domina o terrestre raso
Atenuação (Q)	ausente	amplitude e fase erradas
Anisotropia	ausente	exige VTI/TTI

O que sobrevive bem é a **cinemática** das ondas P, e é por isso que a FWI acústica ainda constrói bons modelos de velocidade mesmo quando a física está incompleta. O preço é que o erro de modelagem restante é absorvido pelo modelo — ponto retomado no Capítulo 13.

Quando o regime acústico é legítimo

Funciona bem em dado marinho com hidrofones (a coluna d'água é literalmente acústica), ângulos moderados, alvo sendo o modelo de velocidade P de grande escala, e ondas de superfície já removidas.

Falha em dado terrestre com *ground roll*, contrastes de v_s relevantes (sal, carbonatos, fundo marinho duro), quando o objetivo é amplitude/AVO, e em meios fortemente atenuantes com banda larga.

2.4 A fonte: wavelet de Ricker

O termo $s(t)$ é a assinatura da fonte. Em trabalho sintético usa-se quase sempre a Ricker, a segunda derivada de uma gaussiana:

$$w(t) = \left[1 - 2(\pi f_0(t - t_0))^2\right] \exp \left[-(\pi f_0(t - t_0))^2\right]. \quad (2.9)$$

f_0 é a frequência de *pico* do espectro — não a máxima, não a média. O atraso $t_0 = 1/f_0$ torna a wavelet praticamente causal, e o valor não é arbitrário: é onde a amplitude truncada em $t < 0$ cai abaixo de 0,1% do pico (17,5% com $t_0 = 0,6/f_0$; 2,1% com $0,8/f_0$; 0,10% com $1,0/f_0$).

Medindo numericamente a banda (Tabela gerada pela aula 01):

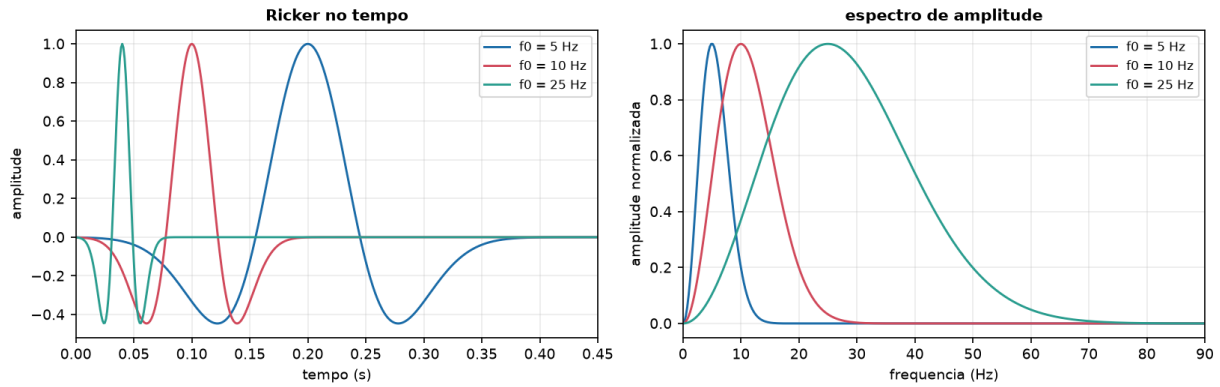


Figura 2.1: Wavelet de Ricker no tempo e na frequência para três valores de f_0 . Aumentar f_0 estreita o pulso no tempo e alarga o espectro. Gerado por aulas/aula01_regime_acustico.py.

f_0 (Hz)	pico medido	$f_{\max}$ (−6 dB)	$/f_0$	$/f_0$ (−20 dB)
5	4,9	8,1	1,61	2,20
10	10,0	16,1	1,61	2,20
15	14,9	24,4	1,63	2,20
25	24,9	40,8	1,63	2,21
40	40,0	65,2	1,63	2,21

As duas colunas servem a propósitos diferentes: a −6 dB ($\approx 1,6f_0$) é a faixa que move a inversão, e serve para raciocinar sobre **resolução**; a −20 dB ($\approx 2,2f_0$) ainda tem energia suficiente para *dispersar* numericamente, e é a que deve ser usada para dimensionar a **malha** — arredondando para $2,5f_0$ por segurança.

$$\lambda_{\min} = \frac{c_{\min}}{f_{\max}}, \quad \text{resolução} \approx \frac{\lambda_{\min}}{2}. \quad (2.10)$$

Capítulo 3

Diferenças finitas: estabilidade e dispersão

[aula 02]

Estabilidade e dispersão são duas restrições independentes. A primeira decide se a simulação *roda*; a segunda, se ela está *certa*.

3.1 O operador espacial

A derivada segunda é aproximada por um estêncil centrado:

$$\frac{\partial^2 f}{\partial x^2} \approx \frac{1}{h^2} \left[c_0 f_i + \sum_{k=1}^N c_k (f_{i+k} + f_{i-k}) \right]. \quad (3.1)$$

Ordem	Pontos	Coeficientes $(c_0, c_1, \dots)$
O(2)	3	-2; +1
O(4)	5	-5/2; +4/3; -1/12
O(8)	9	-205/72; +8/5; -1/5; +8/315; -1/560

A aula 02 verifica numericamente a ordem de convergência em $f(x) = \sin(kx)$. O erro relativo medido, com o mesmo h : $7,5 \times 10^{-3}$ para O(2), $8,9 \times 10^{-5}$ para O(4) e $2,0 \times 10^{-8}$ para O(8).

Ordem maior não é só “mais preciso”: é mais preciso *para o mesmo* h , o que permite usar h *maior* e economizar memória e tempo.

3.2 Estabilidade: a condição CFL

O esquema *leap-frog* é condicionalmente estável. A análise de von Neumann leva a

$$C = \frac{c_{\max} \Delta t}{\Delta h} \leq C_{\lim}, \quad C_{\lim} = \frac{2}{\sqrt{n_{\dim} (|c_0| + 2 \sum_k |c_k|)}}. \quad (3.2)$$

Valores em 2D, medidos pela aula 02: $C_{\lim} = 0,7071$ (O2), $0,6124$ (O4), $0,5546$ (O8).

Duas leituras contraintuitivas

- (a) Ordem espacial *maior* reduz o Δt máximo. O estêncil mais largo amplifica mais os números de onda altos e a condição apertada. Você ganha em Δh e perde em Δt .
- (b) O limite depende de $c_{\max}$. Uma única célula com velocidade alta — sal, basalto, um erro de interpolação — derruba o Δt do modelo inteiro.

A transição não é gradual. A aula 02 mede, com $C_{\lim} = 0,6124$: $C = 0,306$ e $C = 0,551$ ficam estáveis; $C = 0,625$ e $C = 0,704$ produzem NaN. Não existe “um pouco instável”. Use sempre uma margem de 10–20%.

3.3 Dispersão numérica: o erro que parece geologia

No esquema discreto, a velocidade de fase depende do número de onda. As componentes de alta frequência viajam com a velocidade errada e o pulso desenvolve uma cauda oscilatória atrás da frente.

Por que isso é perigoso

A cauda dispersiva *parece sinal*: reverberação, camada fina, *ringing* da fonte. Quem não conhece o efeito interpreta artefato numérico como geologia — e, pior, a FWI tenta explicá-lo com estrutura no modelo.

Teste decisivo: num meio *homogêneo* não há o que refletir. Qualquer oscilação depois do primeiro pulso é erro numérico.

O controle é o número de pontos por comprimento de onda:

$$G = \frac{\lambda_{\min}}{\Delta h} = \frac{c_{\min}}{f_{\max} \Delta h}, \quad G \gtrsim \begin{cases} 15-20 & \text{O(2)} \\ 8-10 & \text{O(4)} \\ 4-5 & \text{O(8)} \end{cases} \quad (3.3)$$

Uma precisão sobre a curva de dispersão

A curva mostrada é a do operador **espacial** isolado (esquema semi-discreto, tempo contínuo). A discretização *temporal* acrescenta um erro próprio, e o detalhe interessante é que ele tem sinal **oposto**: o *leap-frog* adianta a fase, o estêncil espacial atrasa, e os dois se cancelam parcialmente.

Medido para O(4) com $G = 6$: a curva semi-discreta dá 0,9939, enquanto o esquema completo dá 0,9980 ($C = 0,3$) ou 1,0055 ($C = 0,5$). O erro real é *menor* que o previsto pela curva espacial sozinha — ela é uma estimativa conservadora, que é exatamente o que se quer para dimensionar malha.

3.4 Receita de dimensionamento

```

1  1. f_max = 2.5 * f0                # banda conservadora da Ricker
2  2. dh    = c_min / (G * f_max)    # G = 8 (O4) ou 5 (O8)
3  3. dt    = 0.9 * C_lim * dh / c_max # margem de 10%
4  4. nt    = tempo_de_registro / dt
5  5. n_abs >= 0.5 * c_max / (f_min * dh) # ver Capítulo 4

```

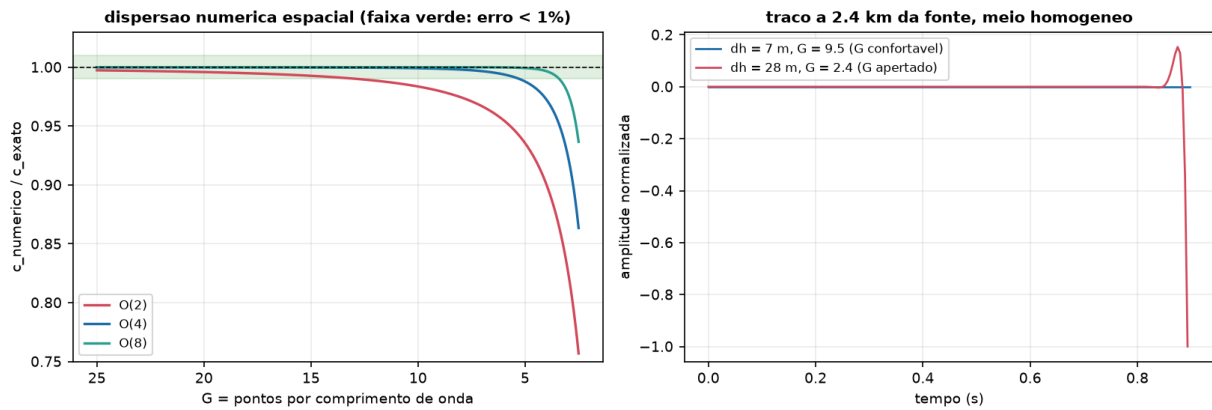


Figura 3.1: Esquerda: dispersão do operador *espacial* por ordem (esquema semi-discreto). Direita: traço a 2,4 km da fonte em meio homogêneo, com $G = 9,5$ e $G = 2,4$. A cauda no traço vermelho não existe na física.

O conflito embutido

Δh é ditado por $c_{\min}$ (menor comprimento de onda) e Δt por $c_{\max}$ (onda mais rápida). Um modelo com grande contraste — água a 1500 e sal a 4500 — é caro dos *dois* lados ao mesmo tempo. É a razão prática de existirem malhas variáveis e esquemas implícitos.

Custo: reduzir Δh pela metade quadruplica o número de pontos em 2D e dobra o número de passos (via CFL). Total: $8\times$ em 2D, $16\times$ em 3D.

Capítulo 4

Bordas, superfície livre e aquisição

[\[aula 03\]](#) [\[aula 04\]](#)

4.1 Por que as bordas não são detalhe

O computador só tem um domínio finito; a Terra é praticamente infinita. Truncar o domínio transforma a borda num espelho.

Em modelagem isso é feio. **Em FWI é pior:** o resíduo passa a conter eventos que a física do modelo nunca vai explicar, e o gradiente tenta explicá-los mexendo na velocidade. Você fabrica estrutura falsa.

Técnica	Como	Custo	Qualidade
Truncar	nada	zero	inaceitável
Cerjan (sponge)	<i>taper</i> multiplicativo	baixo	razoável
Cond. absorvente	equação unidirecional	baixo	boa em inc. normal
CPML	coordenadas complexas	médio	excelente

Medindo de verdade

Para medir a reflexão de borda é preciso uma *referência sem borda*: a mesma simulação num domínio muito maior, onde nada reflete dentro da janela de tempo. A diferença entre os dois traços é, por construção, só artefato.

Cerjan (fwikit)		CPML (PyFWI)	
n_{abs}	reflexão	n_{pml}	reflexão
0	−6,9 dB	0	−11,2 dB
10	−8,5 dB	5	−20,1 dB
20	−13,3 dB	10	−51,5 dB
30	−16,5 dB	20	−69,6 dB
45	−18,4 dB	30	−77,3 dB

A comparação é eloquente: a CPML com 10 pontos já supera o Cerjan com 45. É por isso que códigos de produção usam CPML.

Por que este curso usa Cerjan mesmo assim

O *sponge* de Cerjan é um operador **diagonal**, portanto auto-adjunto. Isso preserva a validade do teste de gradiente do Capítulo 9 sem exigir que você derive o adjunto de uma CPML. Há também um ótimo: *taper* fraco demais não absorve, forte demais cria um degrau de impedância que ele mesmo reflete. O ótimo medido fica em fator $\approx 0,25/n_{abs}$, e o *fwikit* calibra isso sozinho.

Dimensionamento: a moldura precisa de pelo menos meio comprimento de onda da *menor* frequência usada — e como a FWI multiescala começa nas bandas baixas, dimensione por $f_{\min}$, nunca por f_0 .

4.2 Superfície livre: uma escolha física

No topo há uma decisão que não é numérica:

Superfície livre ($p = 0$ em $z = 0$) representa a interface com o ar. A onda reflete com polaridade invertida, e produz as *múltiplas* de superfície e o *ghost* — que são sinais reais do dado.

Borda absorvente no topo representa um meio que continua para cima. Não é físico, mas elimina múltiplas do sintético.

O erro de consistência

Se o dado observado contém múltiplas e o sintético não, esse resíduo inteiro vai para o gradiente. A FWI tenta reproduzir as múltiplas mexendo na velocidade. As saídas são duas: modelar a superfície livre, ou remover as múltiplas do dado antes. Ignorar não é opção.

4.3 Aquisição

A FWI trabalha *sempre* em *shot gathers*, e a razão é conceitual: F simula um tiro por vez, então o resíduo $F(\mathbf{m}) - \mathbf{d}_{obs}$ tem de ser calculado nessa organização. É também por isso que a FWI não usa dado empilhado: o empilhamento descarta a dependência com o *offset*, justamente a informação que separa velocidade de profundidade.

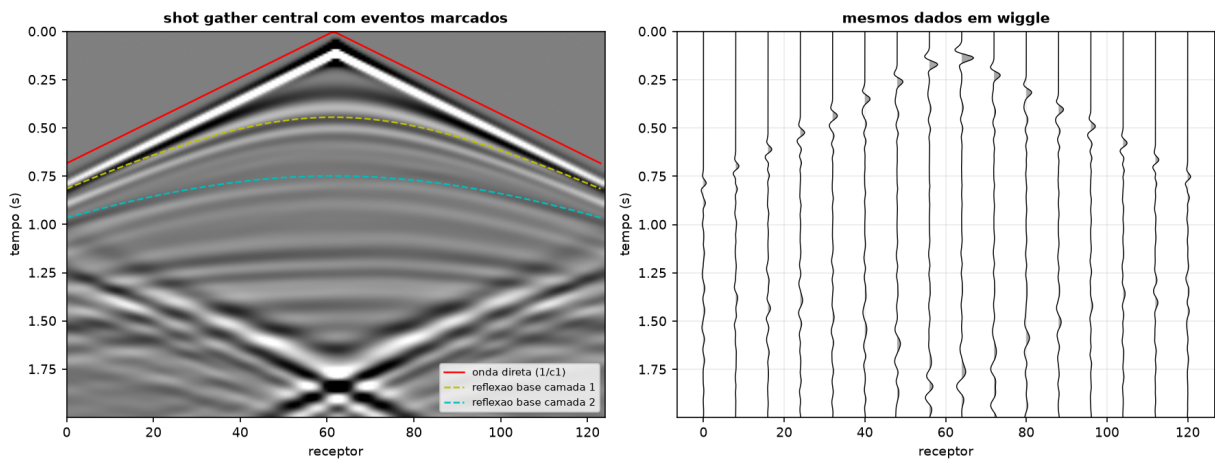


Figura 4.1: Anatomia de um *shot gather*, com as curvas teóricas sobrepostas: onda direta (reta), reflexões (hipérboles).

Anatomia

- **Onda direta** — reta de inclinação $1/c_1$, passa pela origem. Não carrega informação de profundidade; frequentemente é removida.
- **Refrações** (*head waves*) — retas de inclinação $1/c_2$ que cruzam a direta a partir da **distância de cruzamento** $x_{cross} = 2h_1 \sqrt{(v_2+v_1)/(v_2-v_1)}$ — que não se confunde com a *distância crítica* $2h_1 \tan \theta_c$, onde a *head wave* apenas começa a existir e que é bem menor (1006 m contra 2291 m no modelo da aula 04). Dominam o *offset* longo e são a fonte mais rica de informação de velocidade de grande escala.
- **Reflexões** — hipérboles, $t(x)^2 = t_0^2 + x^2/v_{rms}^2$. A curvatura revela a velocidade média.
- **Difrações** — hipérboles de ápice deslocado.

Abertura e profundidade de investigação

$$\text{profundidade confiável} \approx \frac{\text{offset}_{\max}}{3} \text{ a } \frac{\text{offset}_{\max}}{2}$$

A razão é geométrica: para separar velocidade de posição é preciso iluminar cada ponto com *ângulos variados*, e isso exige raios vindo de longe. *Offset* curto só gera incidência quase normal, que é ambígua — mais lento e mais raso produz o mesmo tempo que mais rápido e mais fundo.

Consequência prática: para melhorar a recuperação em profundidade, aumente o *offset*. Receptores mais densos combatem *aliasing*; mais tiros melhoram razão sinal/ruído; frequência maior melhora resolução — mas nenhum deles ilumina mais fundo.

Amostragem espacial

$$\Delta x_{rec} < \frac{c_{ap}}{2f_{\max}} \quad (4.1)$$

Acima disso há *aliasing* espacial: eventos inclinados deixam de ser contínuos. Em FWI isso é especialmente perverso, porque a inversão tenta explicar o padrão serrilhado com estrutura no modelo.

Ruído

A FWI tolera bem ruído *aleatório*: ele não soma coerentemente no gradiente quando se empilham muitos tiros. O que ela *não* tolera é ruído **coerente** e erro de modelagem sistemático — esses somam em fase e viram estrutura.

Parte II — PyFWI

Capítulo 5

PyFWI na prática

[aula 05]

5.1 A anatomia de um script

Todo script de PyFWI tem quatro blocos. Vale memorizar, porque a documentação do pacote é escassa.

```
1  # 1. PARAMETROS -- um dicionario unico
2  inpa = {'ns':1, 'sdo':4, 'fdom':20, 'dh':7, 'dt':0.0005, 't':0.5,
3         'npml':20, 'pmlR':1e-5, 'pml_dir':2, 'acq_type':1, 'device':0}
4
5  # 2. MODELO -- dicionario com 'vp', 'vs', 'rho'
6  model = md.ModelGenerator('louboutin')()
7
8  # 3. AQUISICAO -- posicoes em METROS
9  src_loc, rec_loc = acq.surface_seismic(ns, rec_dis, offsetx, dh, sdo)
10 src = acq.Source(src_loc, dh, dt); src.Ricker(fdom)
11
12 # 4. PROPAGADOR
13 W = wave.WavePropagator(inpa, src, rec_loc, (nz,nx),
14                          n_well_rec=0, chpr=0, components=0)
15 d = W.forward_modeling(model, show=False)
```

Três armadilhas que custam tempo

1. Posições de fonte e receptor são dadas em **metros**, não em índices de malha.
2. `inpa['sdo']` é a ordem espacial *real* (4 ou 8), mas internamente o código faz `self.sdo = sdo/2`.
3. `inpa['acq_type']` seleciona o *kernel* e o mapeamento de receptores: **0** = *crosswell*, **1** = *superfície*, **2** = *ambos*. Os três são válidos — e é justamente por isso que o descuido é perigoso: se o valor não casar com a geometria que você montou, o código *não* levanta erro, mapeia os receptores por outro caminho e devolve um sismograma plausível e errado. Veja a Seção 5.5.

5.2 O parâmetro components

components	Campos	Interpretação
0	taux, tauz (média)	$(\sigma_{xx} + \sigma_{zz})/2 = -p$ — pressão com o sinal trocado
1	taux	só a tensão normal em x
2	vx, vz	velocidade de partícula (geofone)
3	tensor de tensões completo	para inspecionar a física
4	tudo	

O sinal de components=0

O PyFWI devolve a *média das tensões normais*, não a pressão: `acquisition.seismic_section` faz literalmente $(\text{taux} + \text{tauz})/2$. Como $\text{taux} = \sigma_{xx}$ na convenção de tração positiva do *kernel*, isso é $(\sigma_{xx} + \sigma_{zz})/2 = -p$ pela definição (2.5). Para ler pressão, **inverte o sinal**.

Num L2 puramente sintético a polaridade se cancela e o erro passa despercebido; ao comparar com outro código — ou com dado real de hidrofone — ela aparece como correlação negativa.

5.3 A prova numérica do regime acústico

O Capítulo 2 afirmou que a aproximação acústica é a hipótese $\mu = 0$, e que dela decorre um tensor de tensões isotrópico. Podemos *comprovar* isso no próprio PyFWI, que resolve o sistema elástico completo. O experimento: a mesma simulação duas vezes, mudando só v_s .

Caso	$\ \sigma_{xx} - \sigma_{zz}\ /\ \sigma_{xx}\ $	$\ \sigma_{xz}\ /\ \sigma_{xx}\ $
Elástico ($v_s = v_p/\sqrt{3}$)	$3,20 \times 10^{-1}$	$3,91 \times 10^{-1}$
Acústico ($v_s = 0$)	$4,48 \times 10^{-7}$	0,0

Leia estes números

No caso **elástico**, σ_{xx} e σ_{zz} diferem em $\sim 32\%$ e σ_{xz} vale $\sim 39\%$ de σ_{xx} : existe cisalhamento de verdade.

No caso **acústico**, $\sigma_{xx} - \sigma_{zz}$ cai para 10^{-7} — zero até a precisão de `float32` — e σ_{xz} dá **exatamente** 0,0, porque μ multiplica o termo inteiro.

É a demonstração numérica de (2.4) e (2.5). **O PyFWI não tem um solver acústico separado**: ele tem um solver elástico, e o regime acústico se obtém zerando v_s *no modelo* — não apenas pedindo `components=0`, que só escolhe o que é gravado.

5.4 Por que as formas diferem (e não é erro de nenhum dos dois)

A cinemática bate, mas sobrepondo os traços a *forma* do pulso difere. A explicação é exata e já está no Capítulo 2: as duas formulações têm wavelets efetivas que diferem por uma derivada temporal.

Medindo a correlação média sobre todos os receptores:

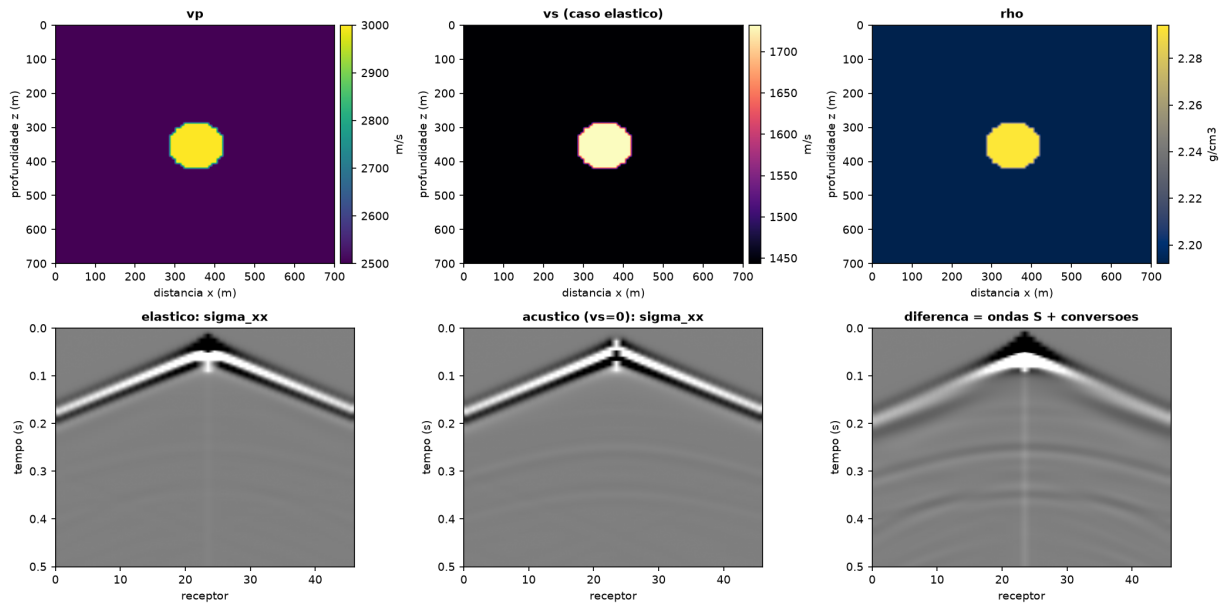


Figura 5.1: Prova do regime acústico. Painel inferior direito: a diferença entre elástico e acústico são as ondas S e conversões P–S — com norma comparável à do próprio dado.

Comparação	Correlação média
traços como saem dos dois códigos	+0,1235
derivando no tempo o traço do fwikit	+0,8501

Sem ajuste e sem parâmetro livre: é exatamente a relação prevista. Os dois códigos estão certos; o que diferia era a **convenção de fonte**, não a física.

Por que a correlação sai *positiva*

Vale fechar a conta dos sinais, porque ela envolve duas inversões que se cancelam. O traço do PyFWI é $(\sigma_{xx} + \sigma_{zz})/2 = -p$ — primeira inversão. E a fonte explosiva do PyFWI (`src_type=0`) soma $+w(t)$ às *tensões* normais, o que em pressão é $-w(t)$, enquanto o `fwikit` soma $+w(t)$ direto na pressão — segunda inversão. As duas se anulam, e a correlação com a derivada temporal sai +0,85 em vez de -0,85.

A lição não muda: *conte os sinais explicitamente*. Aqui eles se cancelaram por sorte; uma comparação em que sobrasse um sinal seria lida como “os códigos discordam” quando o que discorda é a convenção.

As perguntas certas ao comparar dois códigos de onda

Antes de suspeitar da física, verifique a convenção de fonte:

- a formulação é de primeira ou de segunda ordem?
- a fonte entra na pressão, na tensão ou na velocidade de partícula?
- há divisão por Δh^2 (discretização da delta)?

Cada uma dessas escolhas muda a wavelet efetiva — e nenhuma delas aparece no sismograma com uma etiqueta.

5.5 Validação cruzada: como esta comparação achou um bug

Um caso real, ocorrido na montagem deste curso

A comparação entre o PyFWI e o `fwikit` falhou: os dois códigos discordavam em 159 ms, e o `gather` do PyFWI **não tinha ápice** — a curva subia monotonicamente do receptor 0 ao 45, como se a fonte estivesse fora do arranjo.

A causa era `acq_type = 0` com uma geometria de *superfície*. O valor 0 não é inválido — é o código de *crosswell* (`acquisition.acq_parameters`: 0 = crosswell, 1 = superfície, 2 = ambos). O PyFWI então compilou o *kernel* de poço e mapeou os receptores ao longo de uma vertical, sobre posições que na verdade eram uma linha horizontal. Nenhum erro é levantado: as duas configurações são legítimas, só não combinavam entre si.

Duas lições, e a segunda vale mais:

1. Em aquisição de superfície, `acq_type = 1`. Sempre — e confira que ele casa com a função que gerou as posições (`surface_seismic` ↔ 1, `crosswell` ↔ 0).
2. **Um dado sintético errado quase nunca vem com aviso. Ele vem bonito.** A defesa barata é comparar com uma implementação independente e checar se a física elementar aparece — neste caso, se o ápice da *moveout* cai sobre a fonte. Faça isso *antes* de rodar qualquer inversão.

Após a correção, a comparação passa: ápice no receptor 22 ($x = 336$ m) contra 23 ($x = 350$ m), com a fonte em 350 m, e erro rms das primeiras chegadas de 4,4 ms.

O que comparar: escolha a **cinemática**, não a forma do traço. Os dois códigos resolvem discretizações diferentes (velocity–stress intercalado com CPML e densidade variável *versus* pressão colocalizada com Cerjan e densidade constante). Amplitude e fase fina *vão* diferir, e isso não é erro. Tempo de trânsito é propriedade da física, não da discretização.

Capítulo 6

PyFWI por dentro: parâmetros e custo

[aula 06]

6.1 Referência do dicionário `inpa`

Levantada lendo o código-fonte, porque não existe na documentação.

Chave	Obrigatória	O que faz
<code>dh</code>	sim	espaçamento da malha (m)
<code>dt</code>	sim	passo de tempo pedido (veja o reescalonamento)
<code>t</code>	sim	duração do registro (s)
<code>sdo</code>	não (= 4)	ordem espacial <i>real</i> : 4 ou 8; omitir equivale a 4
<code>npml</code>	não (0)	espessura da CPML em pontos
<code>pmlR</code>	com <code>npml</code>	coef. de reflexão teórico (ex. 1e-5)
<code>pml_dir</code>	com <code>npml</code>	0 = só z ; 1 = só x ; 2 = ambas; 3 = ambas sem o topo
<code>acq_type</code>	não (1)	0 = <i>crosswell</i> ; 1 = superfície; 2 = ambos
<code>device</code>	não (0)	índice do dispositivo OpenCL
<code>seimogram_shape</code>	não ('2d')	'2d' $\rightarrow (n_t, n_r n_s)$; '3d' $\rightarrow (n_t, n_r, n_s)$
<code>g_smooth</code>	não (0)	σ da suavização gaussiana do gradiente
<code>energy_balancing</code>	não (False)	normaliza pela iluminação
<code>cost_function_type</code>	não ('l2')	funcional de erro
<code>grad_coeff</code>	não ([1,1,1])	pesos de v_p , v_s , ρ
<code>tv / tikhonov</code>	não	dicionários de regularização
<code>prior_model</code>	não	modelo a priori

Três entradas fáceis de documentar errado

`pml_dir` é invertido em relação à intuição. No código, o valor 0 *zera* a PML em x (sobra só z) e o valor 1 *zera* a PML em z (sobra só x) — o número indica a direção que você *desliga*. O valor 3 mantém as duas mas remove a PML do *topo*, que é como se modela superfície livre.

`sd` não é passo de otimização. É um peso que multiplica os dados *calculados* (`prepare_residual(d_est, sd)`), enquanto os observados entram com peso 1. Deixe em 1,0.

Omitir `sdo` não dá ordem 2. O código faz `self.sdo = sdo/2` e usa 2 como padrão interno, o que corresponde à ordem espacial *real* 4.

O *tipo* que o pacote carrega

A chave é `seimogram_shape`, não `seismogram_shape`. Escrever *certo* faz o PyFWI ignorar a chave em silêncio. Erros assim são a razão de você **sempre** conferir `d.shape` depois da modelagem.

6.2 O reescalonamento automático de Δt

O PyFWI não usa o Δt que você pede:

```
1 self.dt_scale = np.ceil(inpa['dt'] / 0.0006)
2 self.dt       = inpa['dt'] / self.dt_scale
```

Existe um teto rígido de 0,6 ms no passo interno. Pedir $\Delta t = 2$ ms faz o PyFWI subdividir em 4 passos de 0,5 ms.

Três consequências

- Pedir Δt grande **não** economiza computação — só reduz o tamanho do sismograma de saída.
- O teto é calibrado para Δh de poucos metros. Com Δh grande (25–50 m), esse Δt fica desnecessariamente pequeno.
- O PyFWI **não** verifica CFL contra o seu $c_{\max}$. Essa conta continua sendo sua.

6.3 Checkpointing: o compromisso central

Para o gradiente adjunto é preciso correlacionar o campo direto com o adjunto, em cada ponto e cada instante. Mas o direto avança e o adjunto retrocede.

Estratégia	Memória	Computação	Uso
Guardar tudo	$n_t n_z n_x$	1 modelagem	2D pequeno (fwikit)
Recomputar	1 campo	$\sim n_t$	nunca
Checkpointing	$n_{chk} n_z n_x$	~ 2	o padrão (chpr)
Reversão de campo	contorno	2	se não há atenuação

Em 3D isso deixa de ser acadêmico

Um modelo $500 \times 500 \times 300$ com $n_t = 5000$ exigiria **1,5 TB por tiro** para guardar tudo. Por isso todo código de FWI 3D usa checkpointing.

Detalhe pouco mencionado: a *reversão de campo* exige que o operador seja reversível no tempo. Sem atenuação, é. Com atenuação, não — reverter um meio dissipativo amplifica ruído exponencialmente.

6.4 A conta que evita surpresas

$$\text{custo} \approx n_{\text{iter}} \times n_{\text{freq}} \times n_{\text{tiros}} \times 2 \times \text{custo}_{\text{modelagem}} \quad (6.1)$$

O fator 2 é direto + adjunto; a busca linear costuma dobrar de novo.

As duas alavancas que mudam ordens de grandeza

Paralelismo por tiro. Os tiros são independentes — o gradiente é uma *soma* sobre eles. É paralelismo trivial e é a primeira coisa a fazer.

Subconjunto aleatório de tiros por iteração. Em vez de 200 tiros, use 20 sorteados a cada iteração. O gradiente fica ruidoso, mas a direção média está certa e o custo cai 10×.

Parte III — O problema inverso

Capítulo 7

Função objetivo e cycle skipping

[aula 07]

7.1 Formulação

Com F o operador direto e $\mathbf{m}$ o vetor de parâmetros:

$$\mathbf{d}_{calc} = F(\mathbf{m}), \quad (7.1)$$

$$\mathbf{r}(\mathbf{m}) = F(\mathbf{m}) - \mathbf{d}_{obs}, \quad (7.2)$$

$$J(\mathbf{m}) = \frac{1}{2} \|\mathbf{r}\|^2 = \frac{1}{2} \sum_s \sum_r \int_0^T r^2(x_r, t; s) dt. \quad (7.3)$$

A solução iterativa tem sempre a mesma forma:

$$\mathbf{m}_{k+1} = \mathbf{m}_k + \alpha_k \mathbf{d}_k, \quad (7.4)$$

com $\mathbf{d}_k$ a direção de busca (construída a partir do gradiente) e α_k o passo, obtido por busca linear.

Três propriedades de J

Não linear. F depende de $\mathbf{m}$ de forma não linear — a velocidade aparece dentro do operador de onda, e o tempo de trânsito depende dela de maneira integral. Não há solução fechada.

Não convexa. Existem mínimos locais, muitos. Um otimizador local converge para o mais próximo do ponto de partida.

Mal posta. Partes do modelo não são vistas pelos dados. Ali o gradiente é ≈ 0 e o modelo fica onde estava — ou é preenchido por artefatos vindos da regularização.

7.2 Vendo a não convexidade

Num meio homogêneo com um único parâmetro c , é possível varrer c e desenhar $J(c)$ inteiro — coisa impossível com 10^6 parâmetros.

Essa figura contém a ideia mais importante do curso. Em **baixa** frequência a curva é larga e tem um único mínimo: qualquer modelo inicial dentro de uma faixa generosa converge. Em **alta** frequência a curva vira um vale estreito cercado de mínimos locais.

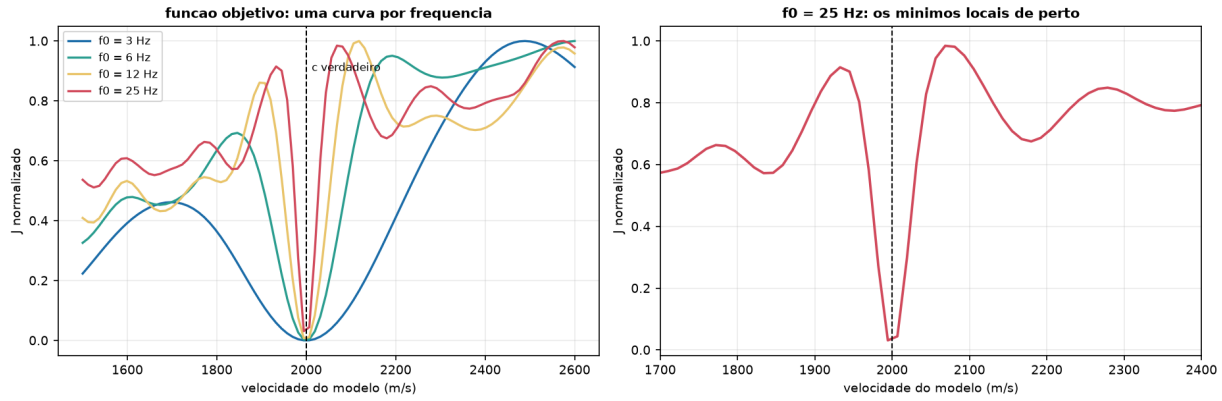


Figura 7.1: $J(c)$ para quatro frequências, meio homogêneo, c verdadeiro = 2000 m/s. Em baixa frequência há um único mínimo largo; em alta frequência, um vale estreito cercado de mínimos locais.

7.3 Cycle skipping: a explicação física

A razão é puramente de *fase*. Se o modelo está errado, o evento sintético chega deslocado de δt em relação ao observado. Para um pulso aproximadamente harmônico de período T :

- $|\delta t| < T/2$: sintético e observado ainda se sobrepõem no *mesmo* ciclo. O resíduo aponta na direção certa e a inversão corrige.
- $|\delta t| > T/2$: o sintético se alinha com o ciclo *vizinho*. O resíduo diminui ao empurrar o modelo para o lado *errado*.

$$|\delta t| < \frac{T}{2} = \frac{1}{2f} \quad (7.5)$$

Para um percurso de comprimento L , $\delta t = L(1/c_{ini} - 1/c_{verd})$, e o critério vira uma tolerância explícita no erro de velocidade:

$$|\delta c| < \frac{c^2}{2fL}. \quad (7.6)$$

f (Hz)	$T/2$ (ms)	$ \delta c _{\max}$ (m/s)	em %
2	250,0	667	33,3
3	166,7	444	22,2
5	100,0	267	13,3
10	50,0	133	6,7
20	25,0	67	3,3
40	12,5	33	1,7

($L = 1500$ m, $c = 2000$ m/s)

A 2 Hz você pode errar 33% e ainda convergir. A 40 Hz precisa começar com menos de 2% de erro — ou seja, precisaria já saber a resposta.

Validação numérica do critério

A aula 07 mede o *vale de atração* de cada frequência — o conjunto de velocidades iniciais que converge para o mínimo certo — e compara com (7.6):

f_0 (Hz)	vale medido (m/s)	semilargura prevista	semilargura medida
3,0	1710 a 2489	422	389
6,0	1858 a 2192	211	167
12,0	1908 a 2118	105	105
25,0	1945 a 2069	51	62

Uma conta de meia linha reproduz a largura medida e a dependência $1/f$ ao longo de uma faixa de $8\times$ em frequência.

Use isso antes de rodar

Estime a frequência mais alta com que pode *começar*, dada a incerteza do seu modelo inicial. Se a tomografia tem 5% de erro e o percurso típico é 2 km, com $c = 2000$ m/s:

$$\delta c = 100 \text{ m/s} \Rightarrow f_{\max} = \frac{c^2}{2\delta c L} = 10 \text{ Hz.}$$

Começar acima disso é desperdício de tempo de máquina.

7.4 A estratégia multiescala

Daí decorre, sem mais nenhuma hipótese, a receita de Bunks *et al.* (1995): inverta primeiro nas frequências baixas, onde o vale é largo; use o resultado como modelo inicial da banda seguinte; repita subindo. Multiescala não é um truque numérico — é a consequência direta de (7.5).

7.5 O modelo inicial

O modelo inicial não precisa ser bonito nem detalhado. Precisa de uma coisa: estar dentro do vale de atração da *menor* frequência utilizável. Isso significa acertar os *grandes* comprimentos de onda, não os detalhes.

Sobre testes sintéticos

Usar o modelo verdadeiro suavizado como inicial — o que `ModelGenerator(...)(smoothing=1)` faz — é um teste **generoso**: ele já começa dentro do vale por construção.

Não está errado usá-lo para validar código. Mas não confunda isso com avaliar a *robustez* do método. Para isso, degrade o modelo inicial de propósito e descubra onde ele quebra. Um experimento que só funciona com inicial privilegiado diz muito pouco sobre dado real.

7.6 Além do L2

Como o problema do L2 é de fase, boa parte da pesquisa consiste em trocar o funcional:

Funcional	Ideia	Preço
L2 clássico	diferença amostra a amostra	cycle skipping
Envoltória	compara envoltórias	perde resolução
Correlação cruzada	penaliza o deslocamento	precisa janelar
Transporte ótimo	distância entre “massas”	custo alto
Adaptativa (AWI)	penaliza o filtro de casamento	mais parâmetros

Ordem de prioridade quando a FWI não converge

1. Baixe a frequência inicial. 2. Melhore o modelo inicial. 3. *Só então* troque o funcional.

Trocar o funcional é a solução mais cara e a que mais introduz parâmetros novos. Ela resolve casos em que 1 e 2 já foram esgotados.

Capítulo 8

O método do estado adjunto

[aula 08]

8.1 Por que não diferenças finitas

O custo da abordagem ingênua é N modelagens *por iteração*:

Modelo	N	modelagens/iter	tempo (a 0,4 s cada)
100×100	10 000	10 000	1,1 horas
200×400	80 000	80 000	8,9 horas
500×1000	500 000	500 000	2,3 dias
3D 300^3	27 000 000	27 000 000	125 dias

Nem em 2D pequeno isso é aceitável.

8.2 A derivação

Trabalhamos com $m(\mathbf{x}) = 1/c(\mathbf{x})^2$, porque a equação é linear nele. O problema direto, como restrição:

$$\mathcal{F}(p, m) \equiv m \frac{\partial^2 p}{\partial t^2} - \nabla^2 p - s = 0. \quad (8.1)$$

Passo 1 — Lagrangiano

A restrição vale para todo $\mathbf{x}$ e todo t ; introduzimos um multiplicador $\lambda(\mathbf{x}, t)$:

$$\mathcal{L} = J + \int_{\Omega} \int_0^T \lambda [m \partial_{tt} p - \nabla^2 p - s] dt d\mathbf{x}. \quad (8.2)$$

Como a restrição é satisfeita pela solução, $\mathcal{L} = J$ — e podemos derivar $\mathcal{L}$ em vez de J , com a vantagem de podermos *escolher* λ para matar o termo difícil.

Passo 2 — variação em relação a p

Integrando por partes duas vezes no tempo (as derivadas passam de p para λ) e duas vezes no espaço (o laplaciano é auto-adjunto):

$$\frac{\delta \mathcal{L}}{\delta p} = [m \partial_{tt} \lambda - \nabla^2 \lambda] + \sum_r r(t) \delta(\mathbf{x} - \mathbf{x}_r). \quad (8.3)$$

Passo 3 — escolher λ

Exigindo $\delta \mathcal{L} / \delta p = 0$, o termo que dependia da variação de p desaparece. Isso *define* a equação adjunta:

$$m \frac{\partial^2 \lambda}{\partial t^2} - \nabla^2 \lambda = - \sum_r r(t) \delta(\mathbf{x} - \mathbf{x}_r) \quad (8.4)$$

com condições **finais** nulas: $\lambda = \partial_t \lambda = 0$ em $t = T$.

Leia a equação adjunta

É a *mesma* equação de onda do problema direto. Mudam duas coisas:

- (1) A fonte não é mais a wavelet no ponto de tiro: é o **resíduo**, com sinal trocado, injetado em *todos* os receptores ao mesmo tempo. Cada receptor vira uma fonte.
- (2) As condições são *finais* : resolve-se de trás para frente.

Interpretação física: você pega o erro que sobrou nos receptores e o joga de volta para dentro da Terra, para ver de onde ele veio.

Como a equação acústica sem atenuação é **auto-adjunta** (invariante por reversão temporal), basta rodar o mesmo propagador ao contrário. Em meios dissipativos essa simetria se perde.

Passo 4 — variação em relação a m

Com λ escolhido, o único termo que sobra é

$$\frac{\partial J}{\partial m}(\mathbf{x}) = \int_0^T \lambda(\mathbf{x}, t) \frac{\partial^2 p}{\partial t^2}(\mathbf{x}, t) dt \quad (8.5)$$

e, somando sobre os tiros, na forma discreta com os pesos de quadratura:

$$\frac{\partial J}{\partial m_i} = \Delta h^2 \Delta t \sum_s \sum_n \lambda_i^n (\partial_{tt} p)_i^n. \quad (8.6)$$

O que essa fórmula é

Uma **correlação cruzada de defasagem zero** entre o campo direto (sua segunda derivada temporal) e o campo adjunto, ponto a ponto.

O gradiente é grande onde os dois campos coincidem no tempo e no espaço — isto é, nos pontos onde uma perturbação do modelo teria afetado o dado.

Essa mesma estrutura é a condição de imagem da migração RTM. FWI e RTM são parentes próximos: a RTM é essencialmente a primeira iteração de uma FWI.

8.3 Da fórmula ao código

```

1  for s in tiros:
2      # (1) DIRETO: resolve e guarda d2p/dt2
3      d_calc, extras = solver.modelar(c, geom, wavelet, i_fonte=s,
4                                     guardar_campo=True)
5
6      # (2) RESIDUO
7      Js, r = misfit_l2(d_calc, d_obs[:, :, s], dt); J += Js
8      # (3) ADJUNTO: mesma equacao, fonte = -residuo, de tras para frente
9      lam = solver.retropropagar(c, geom, r)
10     # (4) GRADIENTE: correlacao cruzada de defasagem zero
11     g_ext += np.einsum("tzz,tzx->zx", lam, extras["dtt"]) * (dt * dh**2)
12
13 # (5) volta ao dominio fisico -- ADJUNTO da extensao de bordas
14 g_m = solver.contrair_adjunto(g_ext)
15 # (6) regra da cadeia para velocidade
16 g_c = g_m * (-2.0 / c**3)

```

O passo (5), que quase ninguém lembra

O modelo físico é estendido com uma moldura absorvente antes de propagar, e essa extensão *replica* os pixels de borda. Isso é um operador linear E , e o gradiente precisa de E^T — a sensibilidade acumulada dentro da moldura tem de ser **somada de volta** sobre o pixel de borda que a gerou, não descartada com um recorte.

Na montagem deste curso esse detalhe valia **9% de erro** na derivada direcional. O Capítulo 9 conta como foi encontrado.

8.4 Como ler um gradiente

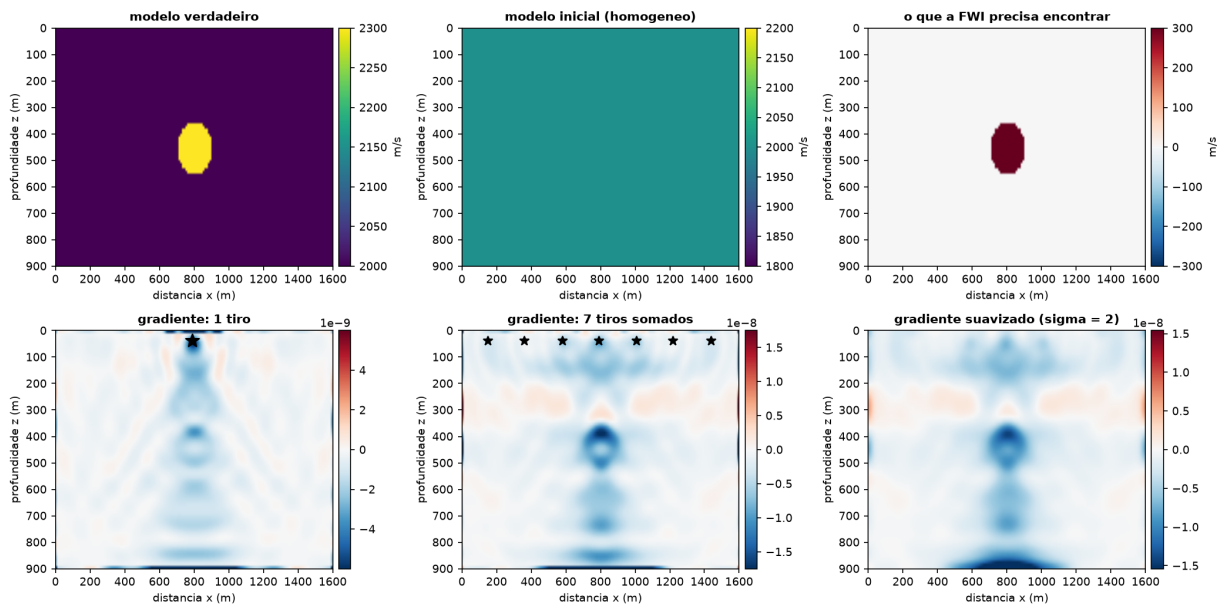


Figura 8.1: Gradiente adjunto: um tiro (esquerda, inferior), sete tiros somados (centro) e suavizado (direita). Um tiro isolado produz arcos de Fresnel ambíguos; a soma sobre tiros é que localiza a anomalia.

1. **O sinal — e em qual parâmetro.** A atualização é $\mathbf{m} \leftarrow \mathbf{m} - \alpha \mathbf{g}$, e o sentido depende do parâmetro em que $\mathbf{g}$ está escrito. Em **velocidade** ($g_c = \partial J / \partial c$), $g_c < 0$ faz c *aumentar*. Em **vagoriedade ao quadrado** ($g_m = \partial J / \partial m$, que é a forma em que (8.5) sai), o sentido se

inverte: como $dm/dc = -2/c^3 < 0$, é $g_m > 0$ que faz a velocidade aumentar. As figuras deste capítulo mostram g_c . Confira sempre o sinal contra uma perturbação conhecida: é o teste de sanidade mais barato.

2. **Artefato perto de fontes e receptores.** O campo tem amplitude enorme ali, e a correlação explode. Não é informação: é geometria.
3. **Os arcos (“banana”).** Um tiro produz uma faixa larga ligando fonte e receptor, não um ponto. É a zona de Fresnel: a onda é sensível a um *volume*. Resolução finita é consequência direta disso.
4. **Iluminação decrescente com a profundidade.** Medida com o PyFWI neste curso: razão raso/profundo de **181**×. Sem correção, a FWI só atualiza a parte rasa.

8.5 Pré-condicionamento

[aula 10]

O que o pré-condicionamento realmente é

A atualização ideal de Newton seria $\delta \mathbf{m} = -H^{-1} \mathbf{g}$, com H a Hessiana — grande demais para ser montada. O pré-condicionador P é uma aproximação barata de H^{-1} .

A diagonal de H_{GN} é bem **aproximada** pela energia do campo direto em cada ponto — a iluminação. A aproximação tem nome e referência: é o *pseudo-Hessiano* de Shin *et al.* (2001), obtido desprezando as contribuições cruzadas entre pontos do modelo.

É uma aproximação, não uma identidade — a diagonal exata de $J^T J'$ envolve também a função de Green até os receptores. Mas é boa o bastante, e barata o bastante, para ser o pré-condicionador padrão. Dividir o gradiente pela energia do campo, portanto, não é um truque cosmético: é uma aproximação diagonal justificada de H^{-1} . É o que o PyFWI faz em `energy_balancing`.

O que ele **não** faz: não corrige cycle skipping, não inventa informação onde não houve iluminação, e não substitui um bom modelo inicial.

Efeito medido (correlação do gradiente com a perturbação verdadeira):

Versão do gradiente	Correlação
bruto	+0,0390
maskado (superfície)	+0,0830
maskado + suavizado	+0,0842
+ compensação de profundidade	+0,0898
<code>energy_balancing</code> + <code>g_smooth</code> (PyFWI)	+0,0880

Como ler esses valores

Os números absolutos são baixos, e isso está *certo*. Um gradiente não é o modelo: é uma direção de busca, espalhada nos arcos de Fresnel e comparada aqui com uma anomalia de ~ 300 pixels em 10 000. Esperar correlação alta seria esperar que a FWI terminasse numa iteração.

O que importa é a variação *relativa*: o tratamento mais que dobra a correlação, e esse ganho, repetido a cada iteração, muda a convergência da inversão inteira.

Receita, na ordem de importância: (1) mascare a faixa de fonte/receptor; (2) compense a iluminação; (3) suavize ($\sigma = 1$ a 3 células); (4) limite o modelo a uma caixa física.

Capítulo 9

Verificação do gradiente

[aula 09]

9.1 Por que este é o teste mais importante do seu código

O modo de falha silencioso

Um gradiente errado **não** gera erro nem trava o programa. Ele gera uma inversão que roda até o fim, produz uma curva de convergência decrescente e devolve um modelo plausível — e errado.

Nenhum dos sintomas normais aparece. Você só descobre quando alguém tenta reproduzir. Por isso a verificação de gradiente é *obrigatória*: é o único teste que separa “o código roda” de “o código está certo”.

9.2 Os três testes

9.2.1 Diferenças finitas pontuais

Compara $\partial J / \partial m_i$ em alguns pixels. Serve para **localizar** onde o gradiente erra. Resultado medido pela aula 09:

Ponto	$ g $ relativo	Adjunto	Dif. finitas	Razão
(0, 50)	1,0	$4,7484 \times 10^1$	$4,7489 \times 10^1$	0,9999
(17, 89)	$5,1 \times 10^{-1}$	$-2,4217 \times 10^1$	$-2,4219 \times 10^1$	0,9999
(34, 89)	$2,1 \times 10^{-1}$	9,8459	9,8452	1,0001
(14, 21)	$8,7 \times 10^{-2}$	-4,1222	-4,1195	1,0006
(38, 27)	$1,0 \times 10^{-7}$	$4,86 \times 10^{-6}$	$4,28 \times 10^{-3}$	0,0011

A última linha é instrutiva: num ponto de gradiente desprezível, numerador e denominador são ambos da ordem do ruído de `float32`, e a razão entre eles não significa nada. **Isso não é erro de gradiente** — é o limite de precisão do próprio teste. Por isso o teste pontual serve para localizar, nunca para aprovar ou reprovar.

9.2.2 Derivada direcional

Compara $\langle \mathbf{g}, \mathbf{h} \rangle$ com a diferença centrada ao longo de $\mathbf{h}$. Testa o gradiente inteiro num número só, e custa 2 modelagens por α . É o teste do dia a dia.

α	Numérico (centrado)	Razão
0,4	$4,56559876 \times 10^{-6}$	1,000661
0,2	$4,56776551 \times 10^{-6}$	1,000186
0,1	$4,56870921 \times 10^{-6}$	0,999980
0,05	$4,56812642 \times 10^{-6}$	1,000107
0,02	$4,57093344 \times 10^{-6}$	0,999493
0,01	$4,56566693 \times 10^{-6}$	1,000646

Erro máximo: 0,07%.

Como diagnosticar a partir do padrão

- **Razão ≈ 1 e estável** $\Rightarrow$ gradiente correto.
- **Razão estável mas $\neq 1$** (por exemplo 0,91 em todos os α) $\Rightarrow$ erro **sistemático**: fator de escala esquecido, sinal trocado, peso de quadratura (Δt , Δh^2) faltando, ou termo ausente no adjunto. Valor estável é boa notícia: o bug é determinístico.
- **Razão instável** $\Rightarrow$ ruído numérico. Reduza a faixa de α ou use precisão dupla.

A escolha da direção $\mathbf{h}$

Uma direção *aleatória* é quase ortogonal ao gradiente (que é suave), o que deixa $\langle \mathbf{g}, \mathbf{h} \rangle$ pequeno e o teste dominado pela curvatura. Use uma direção **suave** — o próprio gradiente normalizado é a escolha mais confiável.

9.2.3 Teste de Taylor

O teste definitivo: não compara valores, compara *taxas de convergência*.

$$E_0(\alpha) = |J(\mathbf{m} + \alpha \mathbf{h}) - J(\mathbf{m})| \sim \mathcal{O}(\alpha), \quad (9.1)$$

$$E_1(\alpha) = |J(\mathbf{m} + \alpha \mathbf{h}) - J(\mathbf{m}) - \alpha \langle \mathbf{g}, \mathbf{h} \rangle| \sim \mathcal{O}(\alpha^2). \quad (9.2)$$

α	E_0	E_1	ordem E_0	ordem E_1
0,4	$2,285 \times 10^{-6}$	$4,578 \times 10^{-7}$	—	—
0,2	$1,028 \times 10^{-6}$	$1,146 \times 10^{-7}$	1,15	2,00
0,1	$4,855 \times 10^{-7}$	$2,867 \times 10^{-8}$	1,08	2,00
0,05	$2,355 \times 10^{-7}$	$7,116 \times 10^{-9}$	1,04	2,01
0,025	$1,160 \times 10^{-7}$	$1,756 \times 10^{-9}$	1,02	2,02

$E_0 \sim \mathcal{O}(\alpha)$ confirma que J é diferenciável e que a perturbação está no regime linear. $E_1 \sim \mathcal{O}(\alpha^2)$ confirma que o gradiente é **exato** — porque só então o termo de primeira ordem é cancelado perfeitamente e sobra a curvatura.

Se E_1 **cair como** $\mathcal{O}(\alpha)$, sobrou termo de primeira ordem: o gradiente não é a derivada correta. É o diagnóstico mais informativo de todos.

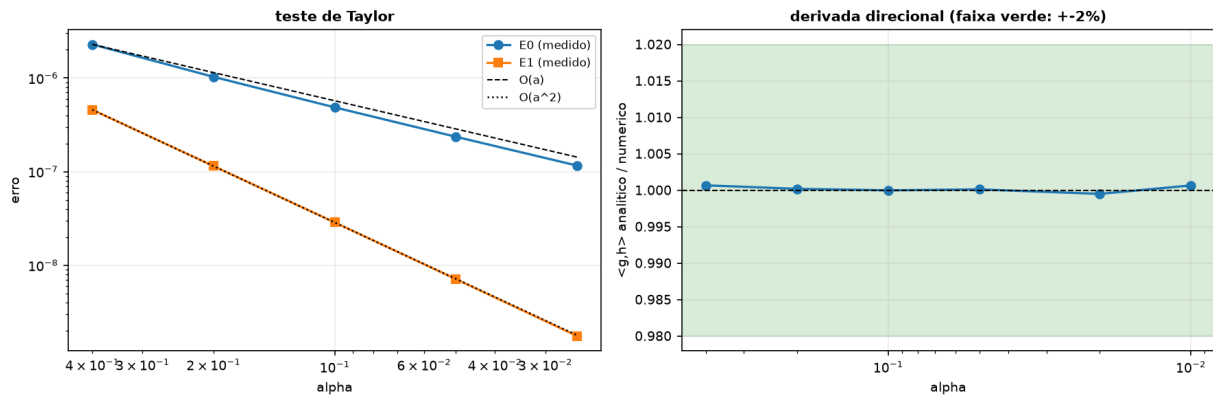


Figura 9.1: Esquerda: teste de Taylor em escala log-log, com as retas de referência $O(\alpha)$ e $O(\alpha^2)$. Direita: razão da derivada direcional (faixa verde: $\pm 2\%$).

9.3 Estudo de caso: o bug que estes testes encontraram

Vale o relato completo, porque mostra os três testes se complementando.

O propagador deste curso **reprovou** no teste da derivada direcional:

α	numérico	razão
0,4	$2,41766 \times 10^{-6}$	0,911797
0,2	$2,41808 \times 10^{-6}$	0,911639
0,1	$2,41747 \times 10^{-6}$	0,911871
0,05	$2,41784 \times 10^{-6}$	0,911728
0,02	$2,41946 \times 10^{-6}$	0,911119
0,01	$2,41739 \times 10^{-6}$	0,911900

O padrão é decisivo: 0,9118 em *todos* os α , ao longo de uma faixa de $40\times$. Perfeitamente estável — o que descarta ruído numérico e aponta para erro sistemático.

O teste **pontual** localizou a região: razões $\approx 0,99$ no miolo do modelo e $\approx 0,72$ perto da superfície. O erro estava nas bordas.

A causa: a extensão do modelo para a moldura absorvente usa `mode='edge'`, que replica cada pixel de borda por toda a moldura — operador linear E . O gradiente estava voltando ao domínio físico com um simples recorte, mas E^T não é recortar, é **somar a moldura de volta**:

```

1 # ERRADO -- descarta a sensibilidade acumulada na moldura
2 g_m = g_ext[topo:topo+nz, n:n+nx]
3
4 # CERTO -- E^T: dobra a moldura de volta sobre as bordas
5 g[topo]      += g[:topo].sum(axis=0)
6 g[topo+nz-1] += g[topo+nz:].sum(axis=0)
7 g[:, n]      += g[:, :n].sum(axis=1)
8 g[:, n+nx-1] += g[:, n+nx:].sum(axis=1)

```

Com a correção, a razão passou de 0,9118 para 1,0001.

A lição generalizável

Toda operação aplicada ao modelo antes de propagar precisa do seu adjunto no caminho de volta do gradiente: extensão de bordas, interpolação entre malhas, suavização, reparametrização, mudança de unidade.

Se você esqueceu de uma, o teste de gradiente acusa — e é o único que acusa.

Parte IV — Otimização, FWI completa e regularização

Capítulo 10

Otimização

[aula 11]

10.1 O panorama

Todos os métodos têm a forma (7.4). O que muda é como se constrói $\mathbf{d}_k$.

Método	$\mathbf{d}_k$	Memória	Em FWI
Newton	$-H^{-1}\mathbf{g}$	N^2	impossível
Gauss-Newton	$-(J'^T J')^{-1}\mathbf{g}$	N^2	só truncado
Descida máxima	$-\mathbf{g}$	N	barato, lento
Grad. conjugado	$-\mathbf{g} + \beta\mathbf{d}_{k-1}$	$2N$	bom custo/benefício
l-BFGS	$-H_{\text{prox}}\mathbf{g}$	$2mN$	o padrão

Com $N = 10^6$, a Hessiana tem 10^{12} entradas: 4 TB em `float32`. Não se monta, não se armazena, não se inverte.

Mas a Hessiana *importa*: na diagonal, a iluminação de cada ponto; fora da diagonal, o acoplamento entre parâmetros. Os métodos quase-Newton existem para capturar parte disso sem montar a matriz. O l-BFGS constrói uma aproximação da *inversa* usando apenas os últimos m pares $(\mathbf{s}_k, \mathbf{y}_k)$, com $\mathbf{s}_k = \mathbf{m}_{k+1} - \mathbf{m}_k$ e $\mathbf{y}_k = \mathbf{g}_{k+1} - \mathbf{g}_k$.

10.2 A busca linear, onde o tempo é gasto

Achar a direção é metade do trabalho; decidir *quanto andar* é a outra. Em FWI cada tentativa custa uma modelagem completa de todos os tiros.

$$J(\mathbf{m} + \alpha\mathbf{d}) \leq J(\mathbf{m}) + c_1 \alpha \langle \mathbf{g}, \mathbf{d} \rangle \quad (\text{Armijo}) \quad (10.1)$$

Retrocesso: tente α , se falhar reduza pela metade. Gasta 5–10 avaliações. **Ajuste parabólico:** já se conhece $J(0)$ e a inclinação $\langle \mathbf{g}, \mathbf{d} \rangle$; com *uma* avaliação extra fica determinada a parábola

$$q(\alpha) = J_0 + \langle \mathbf{g}, \mathbf{d} \rangle \alpha + c \alpha^2, \quad \alpha^* = -\frac{\langle \mathbf{g}, \mathbf{d} \rangle}{2c},$$

cujo mínimo dá o passo. Acerta em ~ 2 avaliações.

O chute inicial do passo

Em FWI o critério com significado físico é: *não altere o modelo em mais de 1–3% de uma vez*. Isso é mais confiável que qualquer estimativa puramente matemática, porque respeita a escala do problema.

10.3 Comparação medida

Problema real de FWI, malha 60×100 , 5 tiros, $f_0 = 8$ Hz, 12 iterações, mesmo pré-condicionamento:

Método	J_{final}/J_0	erro final	ganho	avaliações
Descida máxima	0,1035	3,82%	+19,0%	30
Gradiente conjugado	0,0493	3,29%	+30,1%	28
l-BFGS	0,0419	3,26%	+30,7%	28

(erro do modelo inicial: 4,71%)

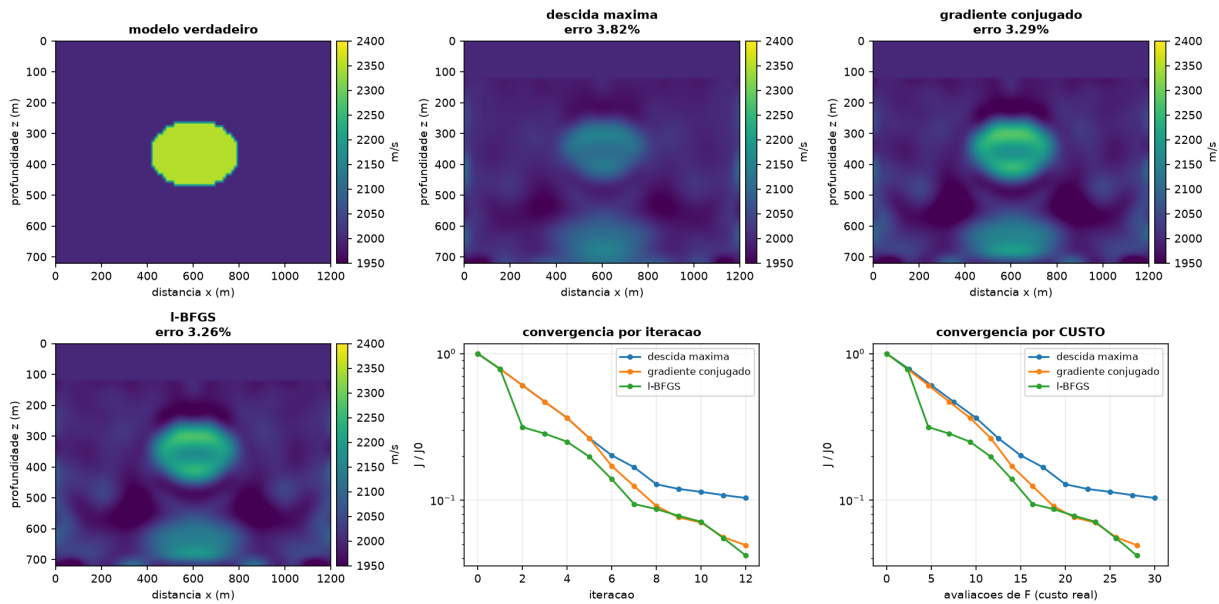


Figura 10.1: Comparação dos três otimizadores. Os dois painéis inferiores à direita contam histórias diferentes: por iteração e por custo.

Compare por CUSTO, não por iteração

Por *iteração*, o l-BFGS quase sempre vence, porque cada iteração usa curvatura acumulada. Por *custo* — número de modelagens, que é o que consome tempo de máquina — a vantagem diminui, porque as iterações do l-BFGS gastam mais avaliações na busca linear. Sempre que comparar otimizadores num relatório, mostre o eixo de custo. Comparar por iteração favorece artificialmente os métodos de iteração cara.

Dois detalhes que resolvem a maioria dos problemas

1. **Reinicie o l-BFGS ao trocar de banda de frequência na multiescala.** Os pares (s, y) descrevem a curvatura do problema *antigo*; mantidos, atrapalham.

2. Se a busca linear falhar ($\alpha = 0$), não insista com mais iterações. Quase sempre significa que a direção não é de descida — gradiente errado, pré-condicionamento exagerado, ou é hora de reiniciar.

Capítulo 11

A FWI completa: multiescala

[aula 12]

11.1 A estratégia

```
1 m = m_inicial
2 para cada banda f em [f_baixa ... f_alta]:
3     filtre d_obs com passa-baixa em f
4     rode algumas iteracoes de FWI partindo de m
5     m <- resultado
```

Cada banda entrega ao estágio seguinte um modelo que já está dentro do vale de atração mais estreito da próxima. O método percorre a não convexidade em degraus.

No PyFWI, `freqs` são as frequências de corte de um filtro Butterworth passa-baixa aplicado a $\mathbf{d}_{obs}$ e $\mathbf{d}_{calc}$ dentro de `cost_seismic`; `iter` é a lista de iterações por banda. Os parâmetros `k_0=1`, `k_end=2` restringem a atualização a v_p , evitando *crosstalk* com ρ .

11.2 Um experimento controlado

Mesmo modelo inicial, mesmo otimizador, *mesmo número total de iterações* (12). A única diferença é a ordem em que a informação entra.

Métrica	Inicial	Mono (25 Hz)	Multi (5/12/25)
Erro relativo	8,00%	7,35%	7,20%
Ganho sobre o inicial	—	+8,1%	+10,0%
Correlação	0,9333	0,9173	0,9296
R^2	0,7284	0,7708	0,7799
Tempo	—	3,8 min	5,7 min

Erro relativo dos dados: 0,2573 (inicial) $\rightarrow$ 0,0719 (final).

Leia este resultado com honestidade

A vantagem da multiescala aqui é real mas **modesta** — 0,15 pontos percentuais. E isso é esperado: o modelo inicial tem apenas 8% de erro e correlação 0,93 com o verdadeiro, ou seja, já está *dentro* do vale de atração de 25 Hz. Quando o monoescala não sofre cycle skipping, não há muito para o multiescala resgatar.

A vantagem da multiescala cresce exatamente quando o modelo inicial piora — e é por isso que o exercício de degradar o inicial de propósito é o que realmente mede o valor da estratégia. Um experimento que só mostra ganho quando o adversário já estava bem não demonstra grande coisa.

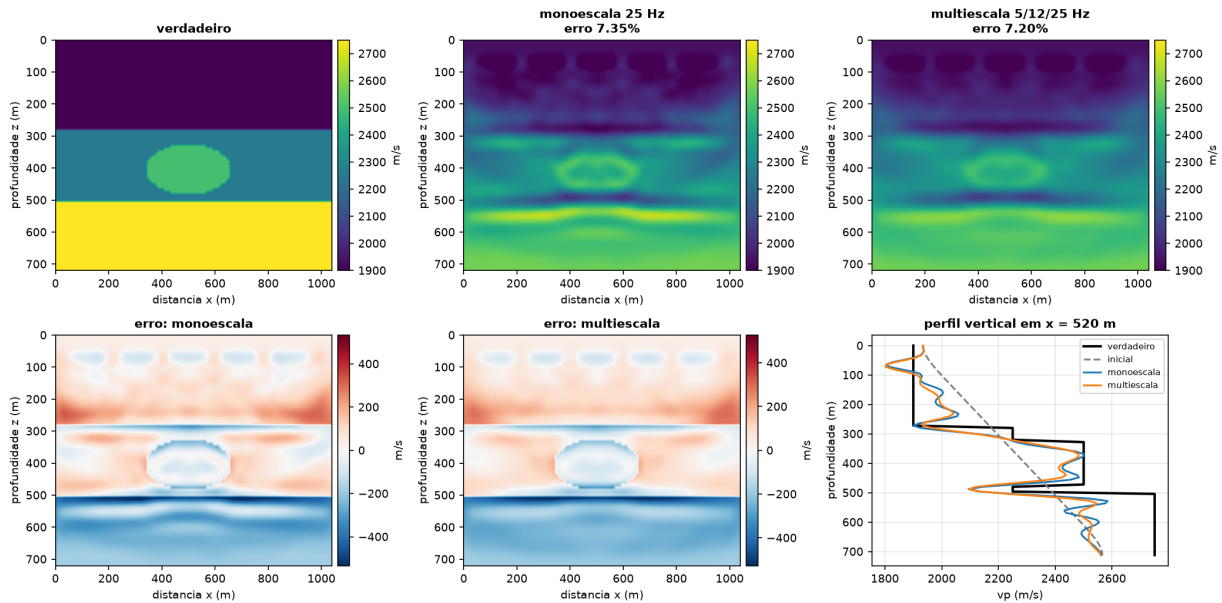


Figura 11.1: Monoescala *versus* multiescala: modelos recuperados, mapas de erro e perfil vertical.

11.3 Estudo de robustez: e quando o inicial é ruim?

O experimento acima parte de um modelo inicial decente, e por isso a vantagem da multiescala fica modesta. O teste que mede o valor real da estratégia é degradar o inicial de propósito. Quatro modelos iniciais, mesmo fluxo, 12 iterações no total em ambos os casos:

Modelo inicial	erro ini.	mono 25 Hz	multi 5/12/25	vantagem
linear suave	7,96%	7,34%	7,07%	+0,27 pp
linear lento (viesado)	16,68%	17,57%	17,34%	+0,24 pp
constante	16,32%	16,10%	16,06%	+0,04 pp
suavizado forte	4,95%	4,01%	3,70%	+0,31 pp

Três leituras, e a segunda é a mais importante

Primeira: a multiescala venceu em *todos* os casos. A vantagem é consistente, ainda que modesta nesta escala de problema.

Segunda — olhe a linha “linear lento”. O erro inicial era 16,68% e a FWI terminou em 17,57% (mono) e 17,34% (multi). Ou seja: **a inversão piorou o modelo**. Ela não travou, não deu erro, e a função objetivo caiu direitinho. Ela simplesmente convergiu para um mínimo local e, no caminho, degradou o que já se sabia.

Este é o fracasso mais perigoso da FWI, porque *não se parece com fracasso*. Olhando só a curva de J e a figura do modelo final, você concluiria que funcionou. É por isso que reportar o erro do modelo **inicial** não é formalidade: sem ele, não há como saber se a inversão ajudou.

Terceira: o melhor resultado absoluto (3,70%) veio do inicial “suavizado forte” — que é o modelo verdadeiro filtrado, o inicial privilegiado que só existe em teste sintético. Compare com “constante”, que termina em 16%: a diferença entre os dois não está no algoritmo, está no que já se sabia antes de começar.

11.4 Como reportar uma FWI

O mínimo que sustenta uma conclusão

Uma figura de “antes e depois” não sustenta nada. Reporte:

- **erro relativo de modelo** $\|\mathbf{m}_{est} - \mathbf{m}_{verd}\| / \|\mathbf{m}_{verd}\|$;
- **curva de convergência** de J , com eixo de *custo*;
- **perfis verticais** comparando verdadeiro, inicial e invertido;
- **mapa de erro** $\mathbf{m}_{est} - \mathbf{m}_{verd}$, que revela *onde* o método falhou;
- **comparação de sismogramas** observado $\times$ final.

E uma exigência de honestidade: **relate o erro do modelo inicial**. Uma FWI que sai de 2% para 1,5% não demonstra quase nada; uma que sai de 20% para 5% demonstra muito. Sem o ponto de partida, o número final não significa nada.

Capítulo 12

Regularização

[aula 13]

12.1 Por que regularizar

A FWI é mal posta em dois sentidos. **Espaço nulo:** existem perturbações que praticamente não mudam os dados — regiões não iluminadas, estruturas menores que $\lambda/2$, combinações de parâmetros que se cancelam. **Amplificação de ruído:** nas direções de baixa sensibilidade, um resíduo pequeno exige perturbação grande do modelo.

$$J_{total}(\mathbf{m}) = J_{dados}(\mathbf{m}) + \lambda R(\mathbf{m}) \quad (12.1)$$

Uma distinção que evita confusão

Regularização **não** melhora o ajuste dos dados. Ela sempre *piora* J_{dados} , por construção. O que ela faz é escolher, entre os muitos modelos que ajustam os dados igualmente bem, aquele mais plausível segundo um critério que *você* definiu.

Isso significa que a escolha de R é uma afirmação sobre a geologia, e precisa ser justificada como tal.

12.2 Tikhonov *versus* Variação Total

$$R_{\text{Tikhonov}}(\mathbf{m}) = \frac{1}{2} \int |\nabla m|^2 \quad (\text{norma } L_2) \quad (12.2)$$

$$R_{\text{TV}}(\mathbf{m}) = \int |\nabla m| \quad (\text{norma } L_1) \quad (12.3)$$

A troca de L_2 por L_1 parece detalhe e não é. Compare o custo de acomodar uma variação total de 1:

	L_2	L_1
Um salto de 1 em um ponto	1,00	1,00
A mesma variação em 10 passos de 0,1	0,10	1,00

A L_2 é *barata* para muitas variações pequenas e cara para um salto grande: por isso ela **suaviza**, borrando interfaces. A L_1 cobra o mesmo nos dois casos: por isso ela tolera saltos nítidos enquanto ainda pune oscilação de ruído. Daí a TV **preservar interfaces**.

Escolha: alvo estratificado com contatos nítidos pede TV; variação gradual pede Tikhonov.

12.3 Um resultado que contraria a expectativa

O experimento da aula 13 usa dado com relação sinal/ruído de **0 dB** (ruído com a mesma energia do sinal), 3 tiros e 18 iterações:

Variante	Erro final	Correlação
modelo inicial	4,75%	0,9554
gradiente cru, sem regularização	3,60%	0,9699
gradiente cru + Tikhonov	4,42%	0,9595
gradiente cru + TV	4,39%	0,9613
só suavização do gradiente	3,56%	0,9709

Por que a regularização explícita perdeu aqui

O modelo inicial já é suave, e com 18 iterações a inversão não tem tempo de gerar estrutura de alta frequência. O erro que sobra **não é rugosidade**: é falta de *contraste* — os degraus de velocidade ainda não foram recuperados em amplitude.

Tikhonov e TV penalizam exatamente o gradiente espacial, ou seja, penalizam o contraste que ainda falta construir. Estão puxando na direção errada *para este problema*.

E note a última linha: apenas suavizando o gradiente, sem termo algum no funcional, chega-se ao melhor resultado da tabela. Uma operação de uma linha.

Regularização implícita

Existe muita regularização em FWI que raramente é reconhecida como tal:

- suavização do gradiente (o `g_smooth` do PyFWI);
- modelo inicial suave — ele já restringe onde a solução pode chegar;
- estratégia multiescala — as bandas baixas impõem suavidade;
- parada antecipada — poucas iterações não produzem alta frequência;
- projeção em caixa.

Somar regularização explícita a tudo isso frequentemente **sobre-regulariza**. O sintoma é o desta tabela: o modelo fica mais suave e o erro *aumenta*.

A conclusão prática não é “não use regularização”, e sim: **meça; diagnostique antes de escolher R** (se o erro residual é rugosidade, Tikhonov/TV ajudam; se é falta de contraste ou resolução, atrapalham).

12.4 Escolhendo λ

Calibre λ de modo que λR valha uma fração conhecida de J_{dados} no ponto inicial (2%, por exemplo). Isso torna a escolha independente de unidades e reprodutível — muito melhor que testar potências de 10 às cegas.

Depois refine pela **curva em L**: para vários λ , ponha J_{dados} em x e R em y , ambos em log. A curva tem forma de L, e o λ bom fica no *cotovelo*.

Uma exigência de honestidade

Em problema *sintético* você conhece o modelo verdadeiro e poderia escolher λ pelo menor erro de modelo. Em dado **real** isso é impossível — você só tem J_{dados} e R . É para isso que existe a curva em L.

Se você reportar resultados escolhendo λ pelo erro de modelo, diga isso explicitamente. Caso contrário o resultado parece melhor do que seria na prática.

12.5 Modelo a priori e vínculos

$$R_{\text{priori}}(\mathbf{m}) = \frac{1}{2} \|W(\mathbf{m} - \mathbf{m}_{\text{priori}})\|^2 \quad (\text{Asnaashari et al., 2013}) \quad (12.4)$$

A matriz W diz *onde* você confia no modelo a priori. Uso clássico: peso alto ao longo de um poço, onde há perfil sônico medido, e zero longe dele.

Vínculo	Como	Quando
Caixa ($v_{\min}, v_{\max}$)	clip após cada passo	sempre
Água fixa	zerar o gradiente na lâmina d'água	dado marinho
Máscara de profundidade	zerar abaixo da penetração	evita artefato
Modelo a priori	$R = \ W(\mathbf{m} - \mathbf{m}_p)\ ^2$	há poço/tomografia
Relação entre parâmetros	vincular v_s, ρ a v_p	multiparâmetro

A projeção em caixa parece grosseira, mas evita o modo de falha mais comum da FWI: um passo ruim empurra a velocidade para valores não físicos, a simulação viola o CFL e a inversão não se recupera.

Onde clidar

Clipe *também dentro* da avaliação de J , e não apenas após a atualização. A busca linear avalia $J(\mathbf{m} + \alpha \mathbf{d})$ **antes** de a projeção ser aplicada, e um passo exagerado pode estourar a simulação ali. Este curso encontrou esse *overflow* exatamente assim.

Parte V — Os limites

Capítulo 13

Os limites da aproximação acústica

[aula 14]

13.1 O inventário de hipóteses

Hipótese	Onde entrou	Quando quebra
$\mu = 0$	Cap. 2	terrestre, contrastes de v_s
densidade constante	Cap. 2	contrastos de impedância
sem atenuação ($Q \rightarrow \infty$)	Cap. 2	meio dissipativo, banda larga
isotropia	Cap. 2	folhelhos, fraturas
2D	curso inteiro	estrutura fora do plano
fonte conhecida	Cap. 4	dado real: wavelet é estimada
ruído gaussiano branco	Cap. 7	ruído coerente, múltiplas

O princípio que vale mais que a tabela

O que a FWI não consegue explicar pela física, ela explica mexendo no modelo. Não existe válvula de escape. Se o dado contém um efeito que o operador não reproduz, esse efeito entra inteiro no resíduo, e o gradiente o converte em estrutura de velocidade. O modelo final absorve o erro de física — e parece um modelo, não um erro.

13.2 Limite 1: densidade e a ambiguidade da impedância

O que controla a reflexão não é a velocidade, é a **impedância** $Z = \rho c$, com $R = (Z_2 - Z_1)/(Z_2 + Z_1)$:

Meio 1	Meio 2	R
$c = 2000, \rho = 2,00$	$c = 2400, \rho = 2,00$	+0,0909
$c = 2000, \rho = 2,00$	$c = 2000, \rho = 2,40$	+0,0909
$c = 2000, \rho = 2,00$	$c = 2400, \rho = 1,67$	+0,0010

A última linha: velocidade 20% maior e densidade 17% menor produzem reflexão praticamente **nula**. Uma FWI acústica de densidade constante interpretaria essa interface como inexistente. É o exemplo mais limpo de ambiguidade velocidade–densidade, e a raiz do *crosstalk* em inversão multiparâmetro.

13.3 Limite 2: elasticidade

Medido no Capítulo 5: a diferença entre o sismograma elástico e o acústico tem norma comparável à do próprio dado. São ondas S e conversões P–S. Essa energia não some do dado real: ou você a modela, ou a remove no processamento, ou ela vai para o resíduo.

Extensão	Parâmetros	Custo	Resolve
Acústica	v_p	$1\times$	cinemática P
Acústica + ρ	v_p, ρ	$1\times$	amplitude de reflexão
Viscoacústica	v_p, Q	$2\text{--}4\times$	dissipação e dispersão
Elástica	v_p, v_s, ρ	$3\text{--}5\times$	ondas S, conversões, AVO
Viscoelástica	$+Q_p, Q_s$	$6\text{--}10\times$	tudo acima
Anisotrópica (VTI)	$+\epsilon, \delta$	$2\text{--}3\times$	folhelhos

13.4 Limite 3: atenuação e o fator Q

O fator de qualidade Q é (aproximadamente) o número de ciclos que uma onda percorre antes de a amplitude cair por $e^{-\pi}$. Valores típicos: água > 1000 ; rocha consolidada 100–300; sedimento não consolidado 20–50; sedimento raso saturado ou com gás 10–30.

Dissipação e dispersão são inseparáveis

Este é o ponto conceitual mais importante da seção, e o mais ignorado.

Um meio que atenua de forma dependente da frequência é **obrigado** a ter velocidade de fase dependente da frequência. A razão é **causalidade**: o sinal não pode chegar antes de ser emitido. Formalmente, as partes real e imaginária da resposta de um sistema causal estão ligadas pelas relações de **Kramers–Kronig**.

Consequência prática: não adianta “corrigir a amplitude” de um dado atenuado e continuar usando uma equação sem perdas. Ao atenuar, o meio também mudou a *fase* — e a FWI é um método governado por fase. Um operador sem dissipação está errado nas duas contas ao mesmo tempo.

O modelo constant- Q de Kjartansson

Entre as representações da dissipação, a de Kjartansson (1979) é a mais usada quando se quer Q aproximadamente constante na banda sísmica. Ela parte de uma lei constitutiva de potência, e o expoente se relaciona a Q por

$$\gamma = \frac{1}{\pi} \arctan\left(\frac{1}{Q}\right), \quad (13.1)$$

com velocidade de fase e amplitude dadas por

$$c(\omega) = c_0 \left(\frac{\omega}{\omega_{ref}} \right)^\gamma, \quad A(\omega) = \exp\left[-\frac{\omega L}{2Q c(\omega)}\right]. \quad (13.2)$$

Precisão da forma usada

O fator de atenuação em (13.2) usa $1/(2Q)$, válido para $Q \gg 1$. A forma exata de Kjartansson traz $\tan(\pi\gamma/2)$ no lugar. A diferença medida é de 0,25% em $Q = 10$ e 0,06% em $Q = 20$ — irrelevante na faixa sísmica, mas vale saber que a expressão é uma aproximação.

No domínio do tempo, essa dependência de potência conduz a operadores de ordem **fracionária** — por isso a implementação viscoacústica é substancialmente mais trabalhosa que a acústica, e costuma ser feita com laplacianos fracionários ou com uma soma de mecanismos de relaxação (modelo SLS).

O que Q faz com um pulso

Aplicando o propagador analítico (13.2) a uma Ricker de $f_0 = 20$ Hz, $c_0 = 2000$ m/s, $L = 1500$ m:

Q	γ	amplitude rel.	f de pico (Hz)
10 000	$3,2 \times 10^{-5}$	1,0000	20,0
200	$1,59 \times 10^{-3}$	0,7738	18,8
100	$3,18 \times 10^{-3}$	0,6008	17,8
50	$6,37 \times 10^{-3}$	0,3713	15,9
20	$1,59 \times 10^{-2}$	0,1056	11,5

Três efeitos simultâneos, nenhum presente na equação acústica: a amplitude cai, a frequência de pico *desce* (as altas são atenuadas mais), e a velocidade de fase efetiva muda.

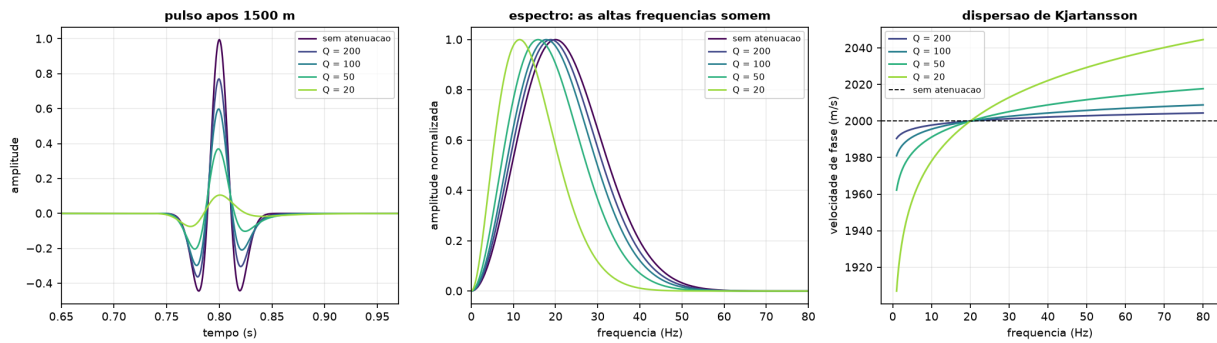


Figura 13.1: Efeito do modelo constant- Q de Kjartansson: pulso no tempo, espectro e dispersão da velocidade de fase.

Quando ignorar Q provoca cycle skipping

Aqui as peças do curso se encaixam. Modelando com c_0 constante mas tendo dado de um meio dispersivo, o erro de tempo de trânsito é

$$\delta t(f) = \frac{L}{c_0} - \frac{L}{c(f)} = \frac{L}{c_0} \left[1 - \left(\frac{f}{f_{ref}} \right)^{-\gamma} \right], \quad (13.3)$$

e o critério de meio ciclo (7.5) decide se isso importa:

Q	L (m)	f (Hz)	$ \delta t $ (ms)	$T/2$ (ms)	
100	1000	10	1,10	50,0	ok
100	6000	60	10,47	8,3	cycle skipping
50	6000	60	20,91	8,3	cycle skipping
20	1000	60	8,66	8,3	cycle skipping
20	6000	10	33,25	50,0	ok
10	6000	10	66,70	50,0	cycle skipping
10	6000	60	102,76	8,3	cycle skipping

($c_0 = 2000$ m/s, $f_{ref} = 20$ Hz)

O padrão é claro: o risco cresce com percurso *longo*, frequência *alta* e Q *baixo*, os três juntos. Com $Q = 200$ e percursos curtos, a aproximação sem perdas é perfeitamente defensável.

O que a tabela não cobre

Ela contabiliza apenas o erro de *fase* por dispersão. O erro de *amplitude* por dissipação é adicional, e afeta diretamente qualquer funcional que compare amplitudes — como o L_2 usado no curso inteiro.

13.5 Como decidir qual física você precisa

A decisão se faz olhando para o dado e para o resíduo, não por gosto nem por disponibilidade de código.

1. **Olhe o resíduo**, não só o valor de J . Resíduo com estrutura *coerente* é a assinatura de física faltando. Resíduo que parece ruído aleatório significa que você chegou ao limite do dado.
2. **Compare espectros** de observado e sintético em função do *offset*. Se o sintético mantém alta frequência que o dado real já perdeu em *offsets* longos, há atenuação não modelada.
3. **Verifique amplitude $\times$ offset**. Decaimento mais rápido no dado real aponta para dissipação ou espalhamento.
4. **Procure eventos ausentes**. Energia no dado real em tempos onde o sintético acústico não produz nada: ondas S, conversões ou múltiplas.
5. **Teste a sensibilidade**. Gere dado sintético com a física mais completa e inverta com a mais simples. O erro de modelo resultante é a medida direta do custo da aproximação — e o experimento mais informativo que existe para justificar (ou dispensar) uma extensão.

O caminho inverso também tem custo

Adicionar física sem necessidade traz mais parâmetros, mais *crosstalk*, mais mínimos locais, mais tempo de máquina, e mais dificuldade em atribuir um resultado a uma causa. Uma FWI acústica bem feita, com bom modelo inicial e multiescala honesta, costuma render mais que uma viscoelástica mal condicionada. A pergunta certa não é “qual a física mais completa?”, e sim “qual é a física mais simples que explica o meu dado?”

Capítulo A

Checklist de diagnóstico

A FWI não converge (J não cai)

1. O gradiente está correto? Rode o teste de Taylor (Cap. 9). **Antes de qualquer outra coisa.**
2. A busca linear está retornando $\alpha = 0$? Direção não é de descida.
3. O sinal do gradiente está certo? Compare $-\mathbf{g}$ com uma perturbação conhecida.
4. O passo inicial é razoável? Deve alterar o modelo em 1–3%.

J cai mas o modelo está errado

1. **Cycle skipping.** Recomece de uma frequência mais baixa. Se o resultado mudar, era isso.
2. Modelo inicial fora do vale de atração — verifique com (7.6).
3. Física faltando: o resíduo final tem estrutura coerente?
4. Bordas: artefatos de reflexão entrando no resíduo.

Só a parte rasa é atualizada

1. Falta máscara na faixa de fonte/receptor.
2. Falta compensação de iluminação (**energy_balancing**).
3. *Offset* insuficiente para a profundidade desejada.

O campo estoura (NaN)

1. CFL: $c_{\max}\Delta t/\Delta h$ acima do limite.
2. O modelo saiu da caixa física durante a busca linear — clipe também *dentro* da avaliação de J .

Cauda oscilatória onde não deveria haver evento

1. Dispersão numérica: G insuficiente. Reduza Δh , aumente a ordem espacial, ou baixe f_0 .

Capítulo B

Notação

Símbolo	Significado
$\mathbf{m}$	vetor de parâmetros do modelo
$m = 1/c^2$	vagarosidade ao quadrado (parâmetro de inversão)
c, v_p	velocidade de onda P (m/s)
v_s	velocidade de onda S (m/s)
ρ	densidade
λ, μ	parâmetros de Lamé (μ = módulo de cisalhamento)
K	módulo de compressibilidade
p	pressão
σ_{ij}	tensor de tensões
F	operador de modelagem direta
J	funcional objetivo
$\mathbf{r}$	vetor residual
$\mathbf{g}$	gradiente
H	Hessiana
$\lambda(\mathbf{x}, t)$	campo adjunto (multiplicador de Lagrange)
α	comprimento do passo
$\Delta h, \Delta t$	espaçamento da malha e passo de tempo
C	número de Courant
G	pontos por comprimento de onda
f_0	frequência de pico da fonte
Q	fator de qualidade
γ	expoente de Kjartansson

Capítulo C

Referências

1. ASNAASHARI, A.; BROSSIER, R.; GARAMBOIS, S.; THORE, P.; VIRIEUX, J. *Regularized seismic full waveform inversion with prior model information*. Geophysics, v. 78, n. 2, p. R25–R36, 2013.
2. BUNKS, C.; SALECK, F. M.; ZALESKI, S.; CHAVENT, G. *Multiscale seismic waveform inversion*. Geophysics, v. 60, n. 5, p. 1457–1473, 1995.
3. CARCIONE, J. M. *Wave Fields in Real Media: Wave Propagation in Anisotropic, Anelastic, Porous and Electromagnetic Media*. 4. ed. Elsevier, 2022.
4. CERJAN, C.; KOSLOFF, D.; KOSLOFF, R.; RESHEF, M. *A nonreflecting boundary condition for discrete acoustic and elastic wave equations*. Geophysics, v. 50, n. 4, p. 705–708, 1985.
5. KJARTANSSON, E. *Constant-Q wave propagation and attenuation*. Journal of Geophysical Research, v. 84, p. 4737–4748, 1979.
6. LOUBOUTIN, M. *et al. Devito (v3.1.0): an embedded domain-specific language for finite differences and geophysical exploration*. Geoscientific Model Development, v. 12, n. 3, p. 1165–1187, 2019.
7. MARDAN, A.; GIROUX, B.; FABIEN-OUELLET, G. *PyFWI: A Python package for full-waveform inversion and reservoir monitoring*. SoftwareX, 2023.
8. NOCEDAL, J.; WRIGHT, S. J. *Numerical Optimization*. 2. ed. Springer, 2006.
9. PLESSIX, R.-E. *A review of the adjoint-state method for computing the gradient of a functional with geophysical applications*. Geophysical Journal International, v. 167, n. 2, p. 495–503, 2006.
10. SHIN, C.; JANG, S.; MIN, D.-J. *Improved amplitude preservation for prestack depth migration by inverse scattering theory*. Geophysical Prospecting, v. 49, n. 5, p. 592–606, 2001.
11. TARANTOLA, A. *Inversion of seismic reflection data in the acoustic approximation*. Geophysics, v. 49, n. 8, p. 1259–1266, 1984.
12. VIRIEUX, J.; OPERTO, S. *An overview of full-waveform inversion in exploration geophysics*. Geophysics, v. 74, n. 6, p. WCC1–WCC26, 2009.
13. VIRIEUX, J.; ASNAASHARI, A.; BROSSIER, R.; MÉTIVIER, L.; RIBODETTI, A.; ZHOU, W. *An Introduction to Full Waveform Inversion*. Society of Exploration Geophysicists, 2017.

Material gerado a partir do código em `aulas/` e `fwikit/`.
Todos os números e figuras são reprodutíveis executando as aulas.